

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Rediseño e implementación de manipuladores para un robot de servicio de propósito general

TESIS

Que para obtener el título de

Ingeniero Mecatrónico

P R E S E N T A N

Rafael Cuéllar Ramírez

Marco Elian Soriano Pimentel

DIRECTOR DE TESIS

Dr. Jesús Savage Carmona



Ciudad Universitaria, Cd. Mx., 2026

Índice general

1. Introducción **5**
- 1.1. Robótica de servicio doméstico . . . 6
- 1.2. Justificación . . . 6
- 1.3. Planteamiento del problema . . . 7
- 1.4. Alcance . . . 7
- 1.5. Objetivos . . . 8
  - 1.5.1. Objetivos específicos . . . 8
- 1.6. Organización del trabajo . . . 8

2. Antecedentes **10**
- 2.1. *Justina* . . . 10
- 2.2. Diseño de manipuladores robóticos . . . 14
  - 2.2.1. Definición . . . 14
  - 2.2.2. Movilidad . . . 14
  - 2.2.3. Criterio de Grashof . . . 15
  - 2.2.4. Síntesis cinemática . . . 16
  - 2.2.5. Análisis cinemático . . . 16
- 2.3. Variables de interés e instrumentación . . . 18
- 2.4. Control de manipuladores robóticos . . . 18

3. Metodología **19**
- 3.1. Identificación de necesidades . . . 20
  - 3.1.1. Recopilación de datos . . . 20
  - 3.1.2. Interpretación y organización de necesidades . . . 20
  - 3.1.3. Importancia relativa y selección . . . 21
- 3.2. Establecimiento de especificaciones objetivo . . . 21
  - 3.2.1. Traducción de necesidades a métricas . . . 21
  - 3.2.2. Benchmarking y análisis competitivo . . . 22
  - 3.2.3. Definición de valores objetivo . . . 23
- 3.3. Generación de conceptos . . . 23
  - 3.3.1. Descomposición funcional del problema . . . 24
  - 3.3.2. Búsqueda externa de soluciones . . . 24
  - 3.3.3. Exploración sistemática y acotamiento del espacio de soluciones 24

3.3.4. Desarrollo de conceptos completos 25

A. Justina **26**
A.1. Planning 26
A.1.1. Paquete `planning_tools` 26
A.1.2. Paquete `clips_node` 27
A.1.3. Paquete `semantic_parser` 27
A.1.4. Paquete `clips_launch` 27
A.2. Supervisor 27
A.2.1. Paquete `supervisor_plan` 28
A.2.2. Paquete `supervisor_execution` 28
A.2.3. Paquete `supervisor_manager` 28
A.2.4. Paquete `supervisor_system` 28
A.2.5. Paquete `surge_et_ambula` 29
A.3. Executor 29
A.3.1. Paquete `executor_tools` 29
A.3.2. Paquete `executor_manager` 31
A.3.3. Paquete `executor_worker` 32
A.3.4. Paquete `gui_controller` 32
A.4. Knowledge 32
A.4.1. Paquete `knowledge_tools` 32
A.4.2. Paquete `knowledge_bank` 32
A.4.3. Paquete `knowledge_manager` 33
A.5. HRI 33
A.5.1. Paquete `hri_tools` 33
A.5.2. Paquete `human_detection` 33
A.5.3. Paquete `human_follower` 34
A.5.4. Paquete `listener` 34
A.5.5. Paquete `talker` 34
A.5.6. Paquete `justina_gui` 35
A.6. Vision 35
A.6.1. Paquete `vision_tools` 35
A.6.2. Paquete `yolo_detect` 35
A.6.3. Paquete `owlv2_detect` 35
A.6.4. Paquete `pc_bbox_pose` 36
A.6.5. Paquete `face_recog` 36
A.6.6. Paquete `table_alignment` 36
A.6.7. Paquete `vacancy_detector` 36
A.7. Navigation 37
A.7.1. Paquete `navigation_tools` 37
A.7.2. Paquete `locations_manager` 37
A.7.3. Paquete `nav_controller` 37
A.7.4. Paquete `navigation_start` 38

A.7.5. Paquete `mvn_pln` . . . 38
A.7.6. Paquete `path_planner` . . . 38
A.7.7. Paquete `simple_move` . . . 38
A.7.8. Paquete `potential_fields` . . . 38
A.7.9. Paquete `map_augmenter` . . . 39
A.7.10. Paquete `augment_gridmap_online` . . . 39
A.7.11. Paquete `mapless_nav` . . . 39
A.7.12. Paquete `debugger` . . . 39
A.8. Manipulation . . . 39
A.8.1. Paquete `manipulation_tools` . . . 40
A.8.2. Paquete `manipulation_planner` . . . 40
A.8.3. Paquete `manipulation_controller` . . . 40
A.8.4. Paquete `move_it` . . . 40
A.8.5. Paquete `trac_ik` . . . 40
A.9. Hardware . . . 41
A.9.1. Paquete `hardware_tools` . . . 41
A.9.2. Paquete `justina_description` . . . 41
A.9.3. Paquete `door_detector` . . . 41
A.9.4. Paquete `scanner` . . . 42
A.9.5. Paquete `justina_hardware` . . . 42
A.9.6. Paquete `base` . . . 42
A.9.7. Paquete `head` . . . 42
A.9.8. Paquete `torso` . . . 43
A.9.9. Paquete `arms` . . . 43
A.9.10. Paquete `grippers` . . . 43
A.9.11. Paquete `joint_states_parser` . . . 43
A.9.12. Paquete `urg_node` . . . 44

B. Protocolo de investigación **45**

C. Carta de consentimiento informado **51**

D. Necesidades identificadas **53**

E. Necesidades seleccionadas **61**

Capítulo 1

Introducción

En 1920, el término **robot**, derivado de la palabra eslava *robota*, que significa trabajo subordinado, fue introducido por el dramaturgo checo Karel Čapek en su obra *Rossum's Universal Robot* [1]. Sin embargo, es hasta 1961 cuando se inicia la robótica como disciplina con la invención de los brazos autónomos para asistir a la manufactura en líneas de producción con el robot *Unimate* [2]. Sus tareas se centraban en la manipulación de piezas y ensambles desde una posición fija en un entorno controlado, prácticamente estático, con la intención de relevar a los trabajadores de tareas monótonas e incrementar la producción.

Debido a esta reciente introducción e investigación en robótica, aún existen diversas definiciones de **robot**, que varían de acuerdo con el campo de aplicación y el área de investigación. Derivado del enfoque industrial inicial, las definiciones tanto de la *International Standard Organization* en el estándar ISO 8373[1] como de la *Robot Institute of America* reducen el término de robot a un manipulador [3]. Por otro lado, Bajd [4] amplía la definición, considerando la robótica contemporánea, como la rama de la ciencia que estudia a aquellos sistemas cuya principal característica es el movimiento.

Durante las últimas décadas se han diseñado y construido una gran cantidad de robots en dos ejes principales: su grado de autonomía, entendida como la habilidad de tomar decisiones basadas en una representación interna del mundo, y su capacidad de actuar en entornos complejos. Ambos relacionados directamente con su inteligencia [2].

De esta forma, la definición que resulta más útil para el desarrollo de este trabajo es la de Arkin [5], quien propone que un robot inteligente es una máquina capaz de extraer información de su ambiente y usar el conocimiento acerca de su mundo para moverse de manera segura y significativa, con un propósito específico.

[1]El estándar ISO 8373 define robot como: "Un manipulador automáticamente controlado, reprogramable, multiuso, programable en tres o más ejes, que pueden estar fijos en un lugar o movilizarse para ser usado en aplicaciones de automatización industrial"; mientras que la RIA considera a un robot como "un manipulador multifuncional reprogramable diseñado para mover materiales, partes, herramientas o dispositivos especializados a través de diversos movimientos programados para la realización de diversas tareas".

1.1. Robótica de servicio doméstico

Los avances exponenciales en robótica han abierto la posibilidad de introducir robots en nuevos entornos. El objetivo de la robótica de servicio es reducir o eliminar el trabajo físico de las personas en lugares como casas. Los dispositivos robóticos tienen el potencial de ayudar a las personas con discapacidad a moverse o brindar compañía a ancianos. Adicionalmente, los robots pueden hacer más seguros los espacios con acciones que van desde recoger objetos para evitar tropiezos hasta verificar que se haya cerrado totalmente la llave de gas [3].

La definición de esta rama más reciente complementa la de Arkin como una máquina móvil reprogramable autónoma o semiautónoma diseñada para operar en entornos dinámicos de manera segura, confiable y robusta, y ser capaz de realizar tareas específicas [3].

Bill Gates [6] vaticinó que en un hogar promedio habría robots de servicio doméstico con propósito específico encargados de cortar el césped, vigilar, limpiar, aspirar, trapear, lavar y planchar, como ya los hay en la actualidad. Pero también habría **robots humanoides de propósito general** que ayudarían a las personas a traer y llevar objetos. Este último tipo de robot tendría la capacidad de comunicarse con los miembros de la casa utilizando lenguaje natural y planear las acciones y movimientos necesarios con base en sus núcleos de acción predefinidos [3] para realizar sus funciones.

De esta forma, los robots de servicio doméstico deben contar con tres habilidades básicas: una navegación robusta, manipulación móvil y comunicación intuitiva con el usuario [7].

En 1997, a partir de avances significativos en inteligencia artificial y robótica, tales como la derrota del campeón mundial de ajedrez Gary Kaspárov a manos del *IBM Deep Blue* y el despliegue del primer sistema robótico autónomo en Marte *Sojourner*, surgió la RoboCup (Robot World Cup), una de las iniciativas con mayor relevancia a nivel internacional en el desarrollo de estos campos. Con el objetivo de definir metas a largo plazo y estandarizar plataformas de desarrollo, se coordinan anualmente las principales instituciones enfocadas en investigación alrededor del mundo para presentar sus avances [8].

Debido al éxito de este proyecto, el alcance de la RoboCup se amplió con el surgimiento de diferentes ligas. RoboCup@HOME se fundó con el objetivo de evaluar las capacidades de la tecnología emergente aplicada a la mejora de las habilidades básicas en robótica de servicio [9].

1.2. Justificación

La manipulación es un problema bien conocido en robótica. Constituye la interfaz de interacción física con objetos y personas, siendo uno de los medios más importantes de un robot para efectuar cambios en su entorno. Las tres principales funciones de una mano humana son explorar, sujetar y manipular objetos. La manipulación robótica se

centra en resolver los últimos dos, ya que el primero cae en el terreno de la háptica [10].

La interacción física presente en nuestra vida diaria cuando se realiza una manipulación, incluso en tareas simples y comunes, va más allá de tomar y colocar algún objeto. Ejemplos de estas tareas son encender la luz, extraer un libro de una estantería, abrir una puerta o cajón, empujar algún objeto, entre otros [11].

Justina es un robot de servicio diseñado en el Laboratorio de Bio-Robótica de la UNAM para operar en entornos domésticos, concebido como parte de una iniciativa para acercar la robótica a las personas y mejorar su calidad de vida [12]. Para resolver la manipulación cuenta con dos brazos de siete grados de libertad cada uno, lo que le permitiría transportar una mayor cantidad de objetos en un menor tiempo y llevar a cabo tareas más complejas.

Contar con dos brazos completamente funcionales abre la puerta a la integración de la manipulación colaborativa. Esto permitiría al robot manejar objetos de mayor tamaño, peso y complejidad, e incorporar acciones como la apertura de contenedores o la manipulación fina de objetos.

1.3. Planteamiento del problema

Los manipuladores de *Justina* exhiben deficiencias críticas que impidieron su participación en la RoboCup@HOME 2025. Los actuadores carecen de la capacidad de carga necesaria para las tareas de manipulación doméstica, los componentes estructurales presentan deformaciones permanentes por sobrecarga y el deterioro acumulado del sistema compromete su desempeño. Asimismo, aunque posee dos brazos, la manipulación colaborativa constituye un desafío técnico pendiente.

En respuesta a esta problemática, el presente trabajo plantea el rediseño e implementación integral de los subsistemas mecánico, electrónico y de software que conforman los brazos de *Justina*.

1.4. Alcance

El presente trabajo abarca el rediseño integral de los brazos manipuladores de *Justina* en tres disciplinas:

- **Mecánica:** diseño de manipuladores y efector finales, análisis por elemento finito y manufactura.
- **Electrónica:** arquitectura del sistema de control, integración de sensores y administración de energía.
- **Software:** desarrollo de nodos de control de los manipuladores y efectores finales e infraestructura para la manipulación colaborativa.

Se contempla además la validación experimental mediante pruebas de desempeño en tareas de manipulación simple en robótica de servicio.

1.5. Objetivos

Diseñar e implementar los subsistemas mecánico y electrónico de los brazos de un robot de servicio de propósito general con el fin de manipular objetos de uso común en un entorno doméstico, así como las interfaces para su integración en el *stack* de software.

1.5.1. Objetivos específicos

- Diseñar y manufacturar dos manipuladores robóticos.
- Diseñar y manufacturar dos efectores finales (EOAT) para manipular objetos comunes de uso doméstico.
- Implementar el nodo de control en ROS para resolver el modelo de la cinemática directa e inversa de la posición de los manipuladores.
- Implementar un nodo de control en ROS para tomar los objetos.
- Actualizar el paquete de ROS con la descripción del robot.

1.6. Organización del trabajo

A continuación se presenta una visión general de la estructura del presente trabajo, ofreciendo una breve descripción del contenido y propósito de cada uno de los capítulos que lo conforman.

En el Capítulo 2 se revisan los fundamentos teóricos y técnicos que sirven de base para el desarrollo del proyecto. Se abordan los temas de mecanismos y manipuladores robóticos, control de posición y trayectoria, medición de par y fuerza en las juntas, así como el uso del *Robot Operating System* (ROS) como plataforma de desarrollo.

En el Capítulo 3 se describe el proceso metodológico seguido para el diseño del sistema. Incluye el levantamiento de necesidades, la generación de especificaciones técnicas, el análisis comparativo de soluciones existentes mediante *benchmarking*, y el desarrollo del diseño conceptual. Asimismo, se detallan las herramientas utilizadas, como la matriz *Quality Function Deployment* (QFD), para la selección y evaluación de alternativas de diseño.

En el Capítulo 4 se presenta el desarrollo integral del sistema desde una perspectiva mecatrónica. El diseño mecánico comprende la estructura del manipulador y el efector final, además de los análisis por elemento finito y el proceso de manufactura. El diseño eléctrico abarca la arquitectura del sistema de control y la administración de energía. Finalmente, se detalla la instrumentación del software y la actualización de los nodos de control del robot *Justina*.

En el Capítulo 5 se describe el protocolo de validación del sistema mecánico y electrónico, así como las pruebas de manipulación implementadas para evaluar el desempeño del robot y sus resultados.

En el Capítulo 6 se presentan las conclusiones derivadas del trabajo, acompañadas de un análisis de los resultados y una discusión sobre los logros alcanzados. También se destacan las principales contribuciones del proyecto y se proponen posibles líneas de trabajo futuro que podrían continuar y mejorar el desarrollo realizado.

Capítulo 2

Antecedentes

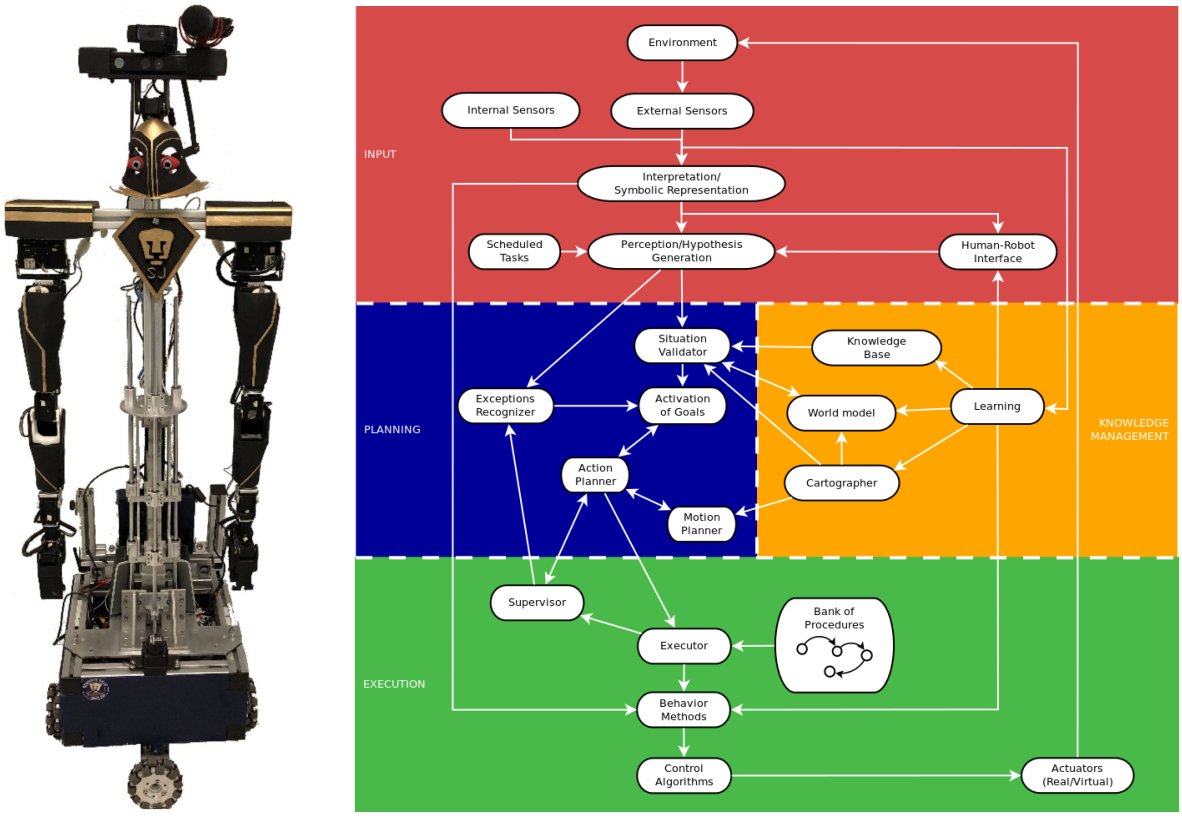
En la Universidad Nacional Autónoma de México (UNAM), el Laboratorio de Bio-Robótica de la Facultad de Ingeniería realiza investigación en áreas como robótica móvil, realidad virtual, reconocimiento de patrones, inteligencia artificial e interfaces hombre-máquina [13]. Fundado en 1996 por Jesús Savage Carmona, el laboratorio comenzó su trayectoria con la adquisición de un robot comercial estadounidense y, desde entonces, ha conformado equipos de estudiantes de licenciatura, maestría y doctorado para desarrollar prototipos propios, tales como TX8, TPR8, PAC-ITO y AL-ITA [14]. Cada plataforma ha sido fruto de una línea de trabajo orientada a la creación de robots de servicio para apoyar a las personas en sus actividades cotidianas en entornos como casas, escuelas, oficinas u hospitales [13].

Actualmente, el robot *Justina* se ha consolidado como la insignia del laboratorio y su referente en la competencia internacional RoboCup@Home. Al constituir el sistema de validación de este trabajo, en el presente capítulo se aborda su arquitectura de hardware y software, para posteriormente exponer la teoría general de manipuladores robóticos, en particular sus variables de interés, instrumentación y estrategias de control.

2.1. *Justina*

Justina (fig. 2.1a) es un robot de servicio doméstico cuyo diseño amigable e inteligente facilita la interacción con los humanos y su entorno. Como proyecto, se enmarca en una iniciativa para mejorar la calidad de vida de las personas, al tiempo que contribuye a la formación de estudiantes en ciencia e ingeniería [12]. La plataforma cuenta con una base móvil omnidireccional, un torso telescópico, un LiDAR, un micrófono cardioide, una cámara RGB-D y dos brazos manipuladores.

La integración del hardware con el software se fundamenta en la arquitectura ViRbot (fig. 2.1b), un sistema modular diseñado para coordinar la operación de robots de servicio autónomos combinando planeación de acciones con comportamientos reactivos sobre una representación dinámica del mundo [15]. Bajo este esquema, las funciones se organizan en cuatro capas: entrada, manejo del conocimiento, planeación y ejecución.



(a) Justina **(b)** ViRbot

Figura 2.1. Arquitectura base del robot *Justina*

Para desempeñar sus tareas, un robot de servicio basado en la arquitectura ViRbot percibe el entorno mediante sus sensores, actualiza su representación del mundo, planifica sus acciones y transmite comandos de control a sus actuadores [16]. En la implementación actual de *Justina*, este modelo conceptual se materializa empleando *Robot Operating System* (ROS), un ecosistema de código abierto cuyo *framework* proporciona herramientas que facilitan la comunicación y sincronización de procesos en el robot [17].

En ROS, las responsabilidades del sistema se estructuran en *nodos*, unidades básicas de cómputo encargadas de realizar una función específica. Los nodos pueden comunicarse entre sí utilizando diversos mecanismos, lo que permite construir sistemas distribuidos en los que sus componentes pueden ejecutarse en distintas arquitecturas de hardware e incluso en diferentes sistemas operativos [18].

A medida que las necesidades de la comunidad superaron el enfoque inicial de ROS, la plataforma evolucionó hacia ROS 2 para contemplar, desde el diseño, nuevos casos de uso, como sistemas embebidos, control en tiempo real y aplicaciones industriales [19]. Para la interacción entre nodos, ROS 2 define tres interfaces de comunicación fundamentales [20]:

- *Tópicos*: Implementan un patrón publicador/suscriptor para la transmisión con-

tinua, unidireccional y asíncrona de datos, como lecturas de sensores o estados del sistema.

- *Servicios*: Implementan un patrón de solicitud/respuesta para interacciones síncronas de corta duración, donde un cliente requiere una respuesta o confirmación inmediata por parte de un servidor.
- *Acciones*: Implementan un patrón de meta/retroalimentación/resultado para tareas asíncronas de larga duración, donde un cliente puede enviar una meta, recibir retroalimentación periódica, cancelar la ejecución y obtener un resultado.

Estas primitivas de comunicación cimentan la arquitectura actual de *Justina*, permitiendo desacoplar el sistema en módulos orquestadores y módulos especializados, cuyos paquetes se documentan en el anexo A. Desde una perspectiva operativa, las tareas del robot provienen de rutinas preprogramadas, como las pruebas de la competencia RoboCup@Home, o de peticiones libres derivadas de comandos de voz. En ambos casos, el procesamiento de la solicitud (fig. 2.2) recae en los módulos orquestadores:

1. *Planning*: Descompone la petición en una secuencia coherente de acciones.
2. *Supervisor*: Examina cada paso del plan generado, proporcionando rutinas de recuperación ante cualquier imprevisto.
3. *Executor*: Traduce las instrucciones de alto nivel en llamadas a los métodos de los módulos especializados correspondientes.

A partir de este punto, la ejecución se distribuye en módulos especializados que proporcionan capacidades específicas:

- *Knowledge*: Administra el estado del entorno para consulta de los demás módulos.
- *HRI*: Canaliza la interacción con personas a través del reconocimiento y síntesis de voz.
- *Vision*: Procesa imágenes para la detección y clasificación de objetos y personas.
- *Navigation*: Localiza el robot y traza rutas hacia el destino evadiendo obstáculos.
- *Manipulation*: Genera y coordina las trayectorias de los brazos y el torso.
- *Hardware*: Abstrae la comunicación con sensores y actuadores.

El desempeño de los módulos *Manipulation* y *Hardware* depende de la fidelidad con la que se representen la geometría y las relaciones cinemáticas del robot. Puesto que los brazos constituyen la interfaz principal con la que *Justina* modifica su entorno, su estudio formal resulta indispensable para traducir acciones de manipulación, como tomar y dejar objetos, en trayectorias concretas.

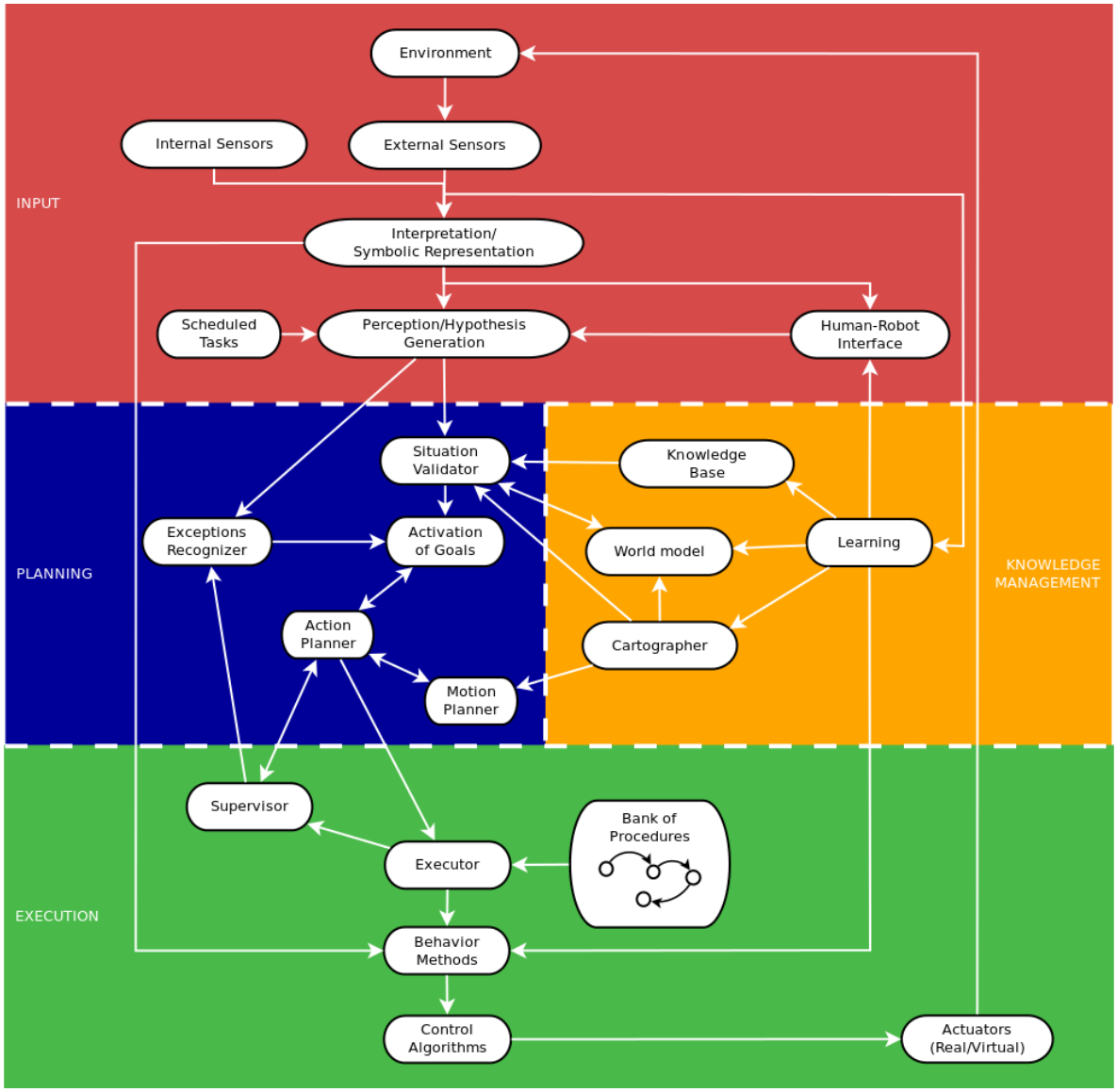


Figura 2.2. Arquitectura de operación de *Justina*.

2.2. Diseño de manipuladores robóticos

La ingeniería mecánica proporciona un conjunto de metodologías para el diseño de manipuladores robóticos, y de máquinas en general. Las ramas predominantes encargadas de ese análisis son la cinemática y la dinámica [21]. La cinemática trata el estudio del movimiento sin considerar las fuerzas que lo ocasionan. La dinámica, por otro lado, se encarga del estudio de las fuerzas sobre sistemas en movimiento. Permite definir los esfuerzos a los que estarán sometidos los elementos del sistema, como función de la aceleración y las fuerzas inerciales [22].

2.2.1. Definición

Un manipulador robótico se define como una cadena cinemática que puede programarse para realizar una amplia variedad de tareas [21]. Una cadena cinemática es un conjunto de eslabones[1] conectados por pares cinemáticos[2], para producir movimientos relativos sincronizados con un propósito determinado [21].

Desde el punto de vista de la teoría de mecanismos [22], un manipulador robótico puede considerarse un mecanismo de varios grados de libertad. A pesar de la definición de Reuleaux [23], que limita un mecanismo a cadenas cinemáticas cerradas y de un grado de libertad, para Norton [22], un mecanismo es una cadena cinemática donde al menos uno de sus eslabones se ha "aterrizado" o fijado al sistema de referencia.

2.2.2. Movilidad

El número de grados de libertad globales de un manipulador determinan su movilidad. Es el número de variables de posición independientes que deben especificarse para conocer la posición de todos los eslabones de la cadena cinemática. La condición de Gruebler considera estos factores para un mecanismo plano [22]. Sin embargo, cuando se tiene más de un eslabón conectado a tierra, el efecto que se obtiene es el de un eslabón más grande y de mayor orden. Además, se debe considerar que las semijuntas únicamente eliminan un grado de libertad en el plano. El resultado es la modificación de Kutzbach [22], mostrada en la ecuación 2.1.

$$M = 3(L-1) - 2J_1 - J_2 \tag{2.1}$$

donde:

M es el número de grados de libertad del mecanismo.

L es el número total de eslabones, incluido el eslabón fijo.

[1] Cuerpo rígido con al menos dos nodos o puntos de conexión. Se clasifican en binarios, ternarios y cuaternarios, dependiendo del número de nodos que posean [22].

[2] Unión de dos o más eslabones en sus nodos, permitiendo movimiento relativo. Se clasifican por sus grados de libertad (GDL) y las más comunes son las juntas completas (1 GDL) y las semijuntas (2 GDL) [22].

J_1 es el número de pares cinemáticos de un grado de libertad.

J_2 es el número de pares cinemáticos de dos grados de libertad.

Finalmente, para mecanismos espaciales, la modificación de Kutzbach se puede extender a eslabones con 6 GDL. De esta forma, se considera que las juntas completas anulan 5 GDL y las de orden inferior van restringiendo 1 GDL a la vez. Tomando en cuenta todos sus efectos se obtiene la ecuación 2.2, donde el subíndice indica el orden de cada tipo de junta.

$$M = 6(L-1) - 5J_1 - 4J_2 - 3J_3 - 2J_4 - J_5 \tag{2.2}$$

2.2.3. Criterio de Grashof

Para el diseño de mecanismos de cuatro barras es necesario determinar qué configuraciones permiten obtener el movimiento requerido. En este sentido, la ley de Grashof establece que para que algún eslabón tenga rotación continua *"la suma de las longitudes del eslabón más corto (s) y más largo (l) no puede ser más grande que la suma de las longitudes de los eslabones restantes (p y q)"* [24], tal como se expresa en la inecuación 2.3.

$$s + l \leq p + q \tag{2.3}$$

Derivado de la condición de Grashof se puede agrupar a las cadenas cinemáticas de cuatro barras planas en tres clases, según se muestra en la figura 2.3. La Clase I cumple con la desigualdad 2.3 y sus inversiones[3] serán la manivela-balancín, la doble manivela y un doble balancín de Grashof, donde el eslabón acoplador hace una revolución completa. La Clase II no cumple con la condición de Grashof y todas sus inversiones serán balancines triples. La Clase III, o caso especial de Grashof, es cuando se cumple la igualdad [22].

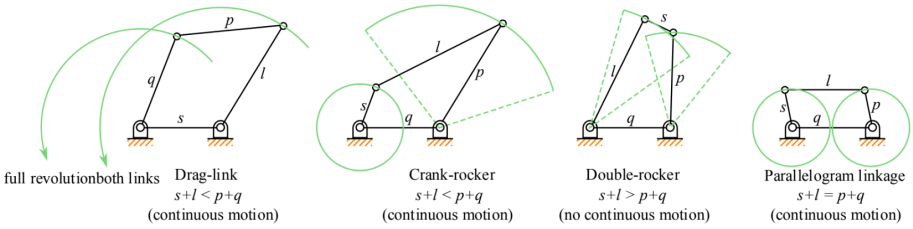


Figura 2.3. Casos del criterio de Grashof [25].

Este criterio constituye una condición preliminar para la síntesis dimensional del mecanismo.

[3]Es la variante de una cadena cinemática dependiendo del eslabón que se designe como "fijo" o de referencia. Esto no cambia el movimiento relativo entre los eslabones, pero sí el movimiento absoluto generado [24].

2.2.4. Síntesis cinemática

La síntesis de un mecanismo garantiza que, por ejemplo, un manipulador serial alcance los puntos de trabajo con las orientaciones deseadas del efector final [25]. Por esta razón, la síntesis cinemática se suele dividir en pasos: síntesis de tipo o topológica, de número y dimensional. Algunos autores suelen juntar la síntesis de tipo y de número en un solo paso, ya que la primera consiste en determinar el tipo de mecanismo que mejor resuelve el problema planteado y el segundo en determinar la cantidad de eslabones y tipo de juntas para obtener la mobilidad requerida. El último paso está dedicado a definir las dimensiones del mecanismo [25].

De acuerdo con Norton [22], las dos grandes clases que engloban las técnicas existentes de síntesis dimensional son la gráfica o geométrica y la analítica. Los métodos gráficos se basan en principios básicos de geometría euclidiana y se centran en cadenas cinemáticas cerradas de cuatro barras. Varían en función de la cantidad de restricciones de diseño, tales como el número de posiciones, de pivotes móviles y fijos.

Una vez que se encuentre una solución potencial se debe evaluar su calidad. El proceso se vuelve entonces un diseño cualitativo mediante análisis sucesivo [22].

2.2.5. Análisis cinemático

El análisis cinemático permite determinar la configuración, velocidad y aceleración de los elementos móviles de un mecanismo a partir de sus relaciones geométricas. Así se garantiza que no fallará en condiciones de operación [22].

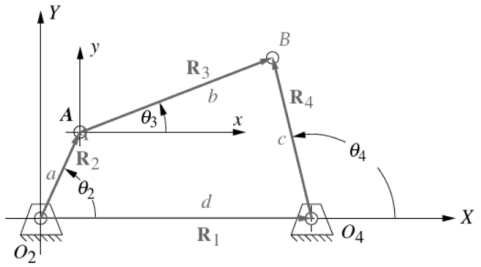
Al considerar el movimiento de un cuerpo rígido se debe partir de la postura o pose, que implica tanto la posición de un punto como la orientación de una línea sobre el eslabón [22], y se expresa con respecto a un marco de referencia[4] asociado a otro eslabón [26]. De esta forma, la pose puede describirse mediante un componente traslacional, dado por el vector de posición ${}^{j}\mathbf{p}_i$, y uno rotacional, dado por la matriz ${}^{j}\mathbf{R}_i$. Con las transformaciones homogéneas, ambos efectos son combinados en una sola representación, como se muestra en la ecuación 2.4 [26]. Además de las matrices de rotación, la orientación puede representarse mediante cuaterniones [26]. En particular, esta representación es utilizada por ROS 2 para describir la orientación tridimensional de los marcos de referencia empleados en sus sistemas de coordenadas [27].

$$ {}^{j}\mathbf{T}_i = \begin{bmatrix} {}^{j}\mathbf{R}_i & {}^{j}\mathbf{p}_i \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{2.4} $$

Para el análisis de posición de un mecanismo plano restringido, éste se representa mediante un lazo vectorial. Los eslabones se representan mediante vectores de posición, cuya suma es igual a cero, como se muestra en la figura 2.4. La notación compleja permite expandir estos vectores mediante la identidad de Euler. De esta forma, se obtiene el sistema de ecuaciones 2.5, con los ángulos θ_3 y θ_4 como incógnitas [22]. Una

[4] Consiste de un origen O_i y una triada de vectores base $(\hat{x}_i, \hat{y}_i, \hat{z}_i)$.

vez obtenida la configuración del mecanismo, las ecuaciones de velocidad se obtienen mediante la diferenciación con respecto al tiempo de las ecuaciones de posición. Una segunda diferenciación permite obtener las ecuaciones de aceleración, utilizando los valores de posición y velocidad correspondientes al instante analizado.



$$\begin{cases} a\cos\theta_2 + b\cos\theta_3 - c\cos\theta_4 - d = 0 \\ a\,\mathrm{sen}\,\theta_2 + b\,\mathrm{sen}\,\theta_3 - c\,\mathrm{sen}\,\theta_4 = 0 \end{cases} \tag{2.5}$$

Figura 2.4. Lazo vectorial de posición de un mecanismo de 4 barras [22].

El mismo procedimiento puede aplicarse al análisis cinemático de un robot serial. Para un manipulador con n grados de libertad, cuando se conocen las variables articulares $\mathbf{q}$ y los parámetros geométricos de los eslabones, es posible resolver el problema de la cinemática directa, mediante el cual se determina la postura del efector final $\mathbf{x}$ con respecto al eslabón base. Esta relación puede expresarse mediante la matriz de transformación homogénea ${}^0\mathbf{T}_n$, resultado de la composición de las transformaciones asociadas a cada junta [26]. Por el contrario, cuando se conoce la pose deseada del efector final, el problema de la cinemática inversa permite determinar las variables articulares correspondientes [26].

A partir de la relación de cinemática directa, su derivada con respecto al tiempo permite establecer la relación entre las velocidades articulares y las velocidades lineal y angular del efector final [26]. Esta relación se expresa mediante el jacobiano, una transformación que relaciona el espacio de juntas con el espacio cartesiano [21], como se muestra en la ecuación 2.6.

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{2.6}$$

Una segunda diferenciación de la ecuación 2.6 permite obtener la relación entre las aceleraciones articulares y las aceleraciones lineal y angular del efector final, expresado en la ecuación 2.7 [28].

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} \tag{2.7}$$

Las relaciones presentadas permiten establecer qué magnitudes deben conocerse durante la operación del sistema. En consecuencia, la selección de los elementos necesarios para su medición constituye el siguiente paso para llevar el modelo cinemático a una implementación física.

2.3. Variables de interés e instrumentación

2.4. Control de manipuladores robóticos

La teoría de control proporciona herramientas para diseñar y evaluar algortimos para realizar los movimientos deseados o las aplicaciones de fuerza.

Capítulo 3

Metodología

Históricamente, el aprendizaje empírico y la experiencia acumulada constituyeron la base del diseño. A partir de la segunda mitad del siglo XX, la importancia económica del desarrollo eficiente y oportuno de productos motivó la creación de procedimientos sistemáticos y flexibles que permitieran planificar, optimizar, verificar y enseñar el proceso de diseño. Tal evolución propició la formalización de metodologías para analizar y descomponer problemas, generar soluciones óptimas y evaluar alternativas bajo criterios técnicos y económicos [29]. Estas metodologías, sin embargo, partían de especificaciones y requisitos que no consideraban a quienes usarían los productos.

El diseño centrado en el usuario (DCU) surgió en el laboratorio de Donald A. Norman en la Universidad de California San Diego como una filosofía que involucra al usuario en todas las fases del desarrollo de un producto, desde su conceptualización hasta su evaluación [30]. Norman propuso en *The Design of Everyday Things* un enfoque que parte de la comprensión profunda de las personas y de las necesidades que se pretenden satisfacer [31]. Este enfoque no es subjetivo, pues se fundamenta en disciplinas científicas como la ergonomía, la psicología y la antropología, y requiere que las decisiones de diseño estén respaldadas por datos obtenidos directamente de los usuarios [32].

En la robótica de servicio doméstico, el DCU resulta ineludible. Para que *Justina* se desenvuelva con éxito en un hogar, debe responder a la dinámica del entorno y de sus habitantes. Identificar sistemáticamente las necesidades de las personas con quienes interactúa es, por lo tanto, una condición fundamental para orientar su desarrollo.

Bajo esta perspectiva, para el rediseño de los manipuladores de *Justina* se adoptó la metodología de Karl T. Ulrich y Steven D. Eppinger, quienes presentan en *Diseño y Desarrollo de Productos* un conjunto de métodos para el diseño de productos físicos, discretos e ingenieriles [33]. Esta obra proporciona un proceso genérico dividido en seis fases: planeación, desarrollo del concepto, diseño a nivel sistema, diseño de detalle, pruebas y refinamiento, e inicio de producción.

Este capítulo abarca únicamente las fases de desarrollo del concepto y diseño a nivel sistema. En ellas se abordan la identificación de necesidades, el establecimiento de especificaciones, la descomposición funcional, la generación de conceptos y la selección de la propuesta que se llevará al diseño de detalle.

3.1. Identificación de necesidades

La identificación de necesidades constituye el punto de partida en la fase de desarrollo del concepto, pues permite definir, de forma independiente a la solución, qué satisface el producto. En este proceso se obtienen tanto las necesidades explícitas, manifestadas directamente, como las necesidades latentes, aquellas que el usuario no logra articular con facilidad pero que representan las mayores oportunidades de innovación [33]. Esta información justifica las especificaciones del producto, haciendo del levantamiento de necesidades el pilar de la metodología.

3.1.1. Recopilación de datos

Para recopilar datos, Ulrich y Eppinger recomiendan las entrevistas, los grupos de enfoque y la observación del producto en uso, desestimando las encuestas escritas en las etapas iniciales de la identificación de necesidades por no aportar información suficiente sobre el usuario [33]. Entre estos métodos, se optó por realizar entrevistas, al considerarse la estrategia más efectiva para conocer la experiencia de interacción con un robot de servicio y las dinámicas del entorno doméstico, sin sesgar la investigación a tareas preconcebidas de manipulación. De esta manera, mediante un muestreo intencional, se concretaron cinco entrevistas individuales y una sesión grupal con cuatro participantes, utilizando guías diferenciadas que se detallan en el Anexo B, orientadas a dos perfiles de usuario:

1. **Desarrolladores de software en robótica de servicio:** Miembros del Laboratorio de Bio-Robótica con experiencia en la competencia RoboCup@Home.

2. **Usuarios finales potenciales:** Personas que requieren asistencia doméstica por autonomía limitada e integrantes de su núcleo familiar.

Las sesiones, aprobadas por el Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería de la UNAM, fueron grabadas en audio y transcritas para su análisis, garantizando la confidencialidad de los participantes conforme al consentimiento informado del Anexo C.

3.1.2. Interpretación y organización de necesidades

Al interpretar los datos recopilados en términos de necesidades del usuario, Ulrich y Eppinger establecen que los enunciados deben redactarse expresando *qué* tiene que hacer el producto y no *cómo* hacerlo, conservando el nivel de detalle de la información original, empleando formulaciones afirmativas, denotando atributos del producto o del usuario, y omitiendo términos como “debe” o “debería” para evitar asignar prioridades de forma prematura [33]. Bajo estas directrices, a partir de los datos obtenidos en las entrevistas se generaron **690 enunciados**.

Tras eliminar frases redundantes, los enunciados se sintetizaron en **122 necesidades estandarizadas**, cuya agrupación por similitud conceptual superó el umbral de 20 grupos sugerido por Ulrich y Eppinger [33] y se estructuró en tres niveles jerárquicos encabezados por **14 necesidades primarias**. La jerarquía completa y sus frecuencias se reportan en el Anexo D.

3.1.3. Importancia relativa y selección

Priorizando la precisión de la consulta directa frente a la velocidad que ofrece el consenso interno [33], la importancia de las necesidades primarias se evaluó mediante una encuesta a los tres usuarios con la mayor riqueza de enunciados. Posteriormente, sus respuestas se integraron en un indicador ponderado, cuya formulación y tablas se presentan en el Anexo E.

Para responder a las expectativas de los usuarios, se seleccionaron las necesidades primarias con una importancia relativa superior al 10 %. Dado que la necesidad *El robot manipula artículos domésticos* triplicó dicho umbral, se decidió trabajar también con sus necesidades secundarias, conformando el conjunto final:

1. **El robot manipula artículos domésticos.**
    - El robot planea trayectorias del manipulador.
    - El robot toma artículos con la fuerza necesaria.
    - El robot extiende el alcance efectivo de sus manipuladores.
    - El robot manipula equipo para realizar tareas domésticas.
2. **El robot es robusto.**

3.2. Establecimiento de especificaciones objetivo

Las necesidades identificadas, aunque útiles para comprender los problemas de interés para el usuario, dejan margen a la interpretación. Las especificaciones, en cambio, explican lo que el producto tiene que hacer a través de una métrica y un valor con sus respectivas unidades. Estas se establecen en al menos dos ocasiones, inicialmente como especificaciones objetivo, formuladas antes de conocer las restricciones tecnológicas y económicas, y más adelante como especificaciones finales, refinadas tras la selección del concepto [33].

3.2.1. Traducción de necesidades a métricas

Para cada necesidad del cliente se identificó la característica precisa y medible del producto que mejor refleja el grado en que esa necesidad queda satisfecha. La relación entre necesidades y métricas se documentó en una matriz de correlación (Cuadro 3.2)

que evidencia qué métricas cubren cada necesidad y permite identificar aquellas que requieren múltiples indicadores. A partir de las necesidades identificadas se definieron las ocho métricas presentadas en el Cuadro 3.1.

Tabla 3.1. Lista de métricas derivadas de las necesidades del cliente.

#	Métrica	Nec. relacionadas	Importancia	Unidad
1	Capacidad de carga	1, 5, 6	5	[kg]
2	Fuerza de agarre	1, 3, 5	5	[N]
3	Alcance	2, 4, 5	5	[mm]
4	Apertura	1, 5	4	[mm]
5	Grados de libertad	4, 5	4	[1]
6	Repetibilidad	2, 5, 6	4	[mm]
7	Grado de protección IP	5, 6	3	[1]
8	Mapa de desensamble	6, 7	3	[s]

Tabla 3.2. Matriz de correlación entre necesidades del cliente y métricas.

Necesidad	M1	M2	M3	M4	M5	M6	M7	M8
El robot manipula artículos domésticos	×	×		×				
El robot planea trayectorias del manipulador			×			×		
El robot toma artículos con la fuerza nec.		×						
El robot extiende el alcance efectivo			×		×			
El robot manipula equipo para tareas dom.	×	×	×	×	×	×	×	
El robot es robusto	×					×	×	×
El robot tiene un diseño intuitivo								×

3.2.2. Benchmarking y análisis competitivo

Para establecer valores de referencia se realizó un análisis comparativo de sistemas representativos de dos categorías: *cobots* de uso industrial —Panda/Franka Emika, Gen3/Kinova, UR3e/Universal Robots, EBLM/Ti5 Robot y OpenArm— y manipuladores de robots de servicio —Stretch 3/Hello Robot, HSR/Toyota, TIAGo/PAL Robotics y G1/Unitree—. Para los efectores finales se compararon grippers de dos dedos (Franka Hand, 2F-85 y 2F-140/Robotiq, HSR) y de tres dedos (3F-155/Robotiq, 3F-AR/UW-Milwaukee, DG-3F-B/Tesollo, Dex3s/Unitree).

La evaluación se realizó mediante un análisis de calidad de función (QFD) que ponderó cada métrica según su importancia relativa a las necesidades primarias identificadas. Para el brazo se evaluaron cuatro grupos de necesidades: planeación de trayectorias

(alcance y repetibilidad), alcance efectivo (alcance y grados de libertad), tareas domésticas (capacidad de carga, alcance, grados de libertad, repetibilidad e IP) y robustez (capacidad de carga, repetibilidad e IP). Para el gripper se evaluaron las necesidades de manipulación (capacidad de carga, fuerza de agarre, apertura, grados de libertad y repetibilidad), control de fuerza, tareas domésticas y robustez.

3.2.3. Definición de valores objetivo

Con base en el análisis de referencia se establecieron valores objetivo para cada métrica en dos niveles: el **valor ideal**, que representa el mejor desempeño razonablemente alcanzable, y el **valor marginal**, que constituye el mínimo aceptable para que el diseño sea viable. El Cuadro 3.3 presenta el conjunto completo de especificaciones objetivo.

Tabla 3.3. Especificaciones objetivo para los manipuladores de Justina.

#	Métrica	Imp.	Unidad	V. marginal	V. ideal
1	Capacidad de carga	5	[kg]	> 2	> 3
2	Fuerza de agarre	5	[N]	> 70	> 100
3	Alcance	5	[mm]	> 500	> 700
4	Apertura	4	[mm]	> 100	> 140
5	Grados de libertad	4	[1]	≥ 5	≥ 6
6	Repetibilidad	4	[mm]	$< \pm 1$	$< \pm 0.1$
7	Grado de protección IP	3	[1]	IP44	IP55
8	Mapa de desensamble	3	[s]	—	—

La métrica M8 (mapa de desensamble), correspondiente a la necesidad de diseño intuitivo, se retiene en el conjunto como requerimiento cualitativo: su naturaleza no admite una cuantificación directa en esta etapa y su evaluación se formalizará durante la fase de selección de conceptos.

3.3. Generación de conceptos

La generación de conceptos es la actividad mediante la cual el equipo de desarrollo explora de manera sistemática el espacio de alternativas de diseño posibles antes de comprometerse con una solución particular. Ulrich y Eppinger [33] definen el concepto de un producto como una descripción aproximada de la tecnología, los principios de funcionamiento y la forma del producto. El método estructurado propuesto consta de cinco pasos: aclarar el problema mediante descomposición funcional, buscar externamente conceptos existentes, generar internamente nuevas soluciones, explorar sistemáticamente las combinaciones posibles y reflexionar sobre los resultados. Dado que la generación de conceptos es relativamente económica en comparación con las fases posteriores del

desarrollo, los autores argumentan que no existe justificación para limitarla, y que un equipo eficiente debería explorar el espacio de alternativas con amplitud suficiente como para tener confianza en que no ha pasado por alto soluciones superiores.

3.3.1. Descomposición funcional del problema

El primer paso consistió en representar el sistema de manipulación como una caja negra que opera sobre flujos de energía, material y señales, para luego dividirla en subfunciones que describen lo que los elementos del manipulador deben hacer. De la totalidad de subfunciones identificadas se seleccionaron los **seis subproblemas críticos** que determinan en mayor medida el desempeño del sistema: aceptar energía, convertir energía a movimiento, transmitir movimiento y fuerza, sensar posición, sensar fuerza y sostener el objeto. El diagrama funcional completo, que muestra la interacción entre estas subfunciones a través de los flujos de energía y señal, se presenta en la Figura

3.3.2. Búsqueda externa de soluciones

Para cada uno de los seis subproblemas críticos se realizó una búsqueda exhaustiva de soluciones existentes en la literatura técnica, catálogos de fabricantes, patentes y en el análisis directo de los robots estudiados durante el benchmarking. Las soluciones encontradas se organizaron en tablas morfológicas —una por subproblema— que cubren los principales principios de solución para cada función. Las tablas completas se incluyen en el Anexo [W].

3.3.3. Exploración sistemática y acotamiento del espacio de soluciones

La combinación irrestricta de las soluciones identificadas genera un espacio de diseño de gran magnitud. Mediante un script de Python se enumeraron sistemáticamente **1943 combinaciones** que representan el espacio de búsqueda completo antes de aplicar criterios de compatibilidad.

Para reducir este espacio a candidatos viables se procedió en dos etapas. Primero, se acotó el conjunto de opciones por subproblema a las soluciones más pertinentes para el contexto de aplicación: robot de servicio doméstico, operación eléctrica e integración con el stack de software existente del laboratorio. Segundo, sobre este espacio reducido se aplicó un criterio de **afinidad entre soluciones**: se conservaron únicamente aquellas combinaciones que representan configuraciones lógicamente coherentes y directamente implementables, que no requieren adaptaciones innecesarias para funcionar conjuntamente. Este filtrado redujo el espacio de candidatos a **384 combinaciones realistas**.

3.3.4. Desarrollo de conceptos completos

Del conjunto de 384 candidatos se seleccionaron **4 conceptos representativos** para su desarrollo en detalle, buscando cubrir la diversidad de enfoques presentes en el espacio de soluciones. Cada concepto se materializó en un modelo CAD conceptual que permitió evaluar de manera preliminar las métricas establecidas en las especificaciones objetivo y facilitar la comparación estructurada en la fase de selección. Los cuatro conceptos, sus características principales y su evaluación frente a las especificaciones se describen en el capítulo siguiente.

Apéndice A

Justina

La arquitectura de software de *Justina*, robot de servicio doméstico del Laboratorio de Bio-Robótica de la UNAM, se implementa sobre ROS 2 y organiza sus paquetes en dos categorías de módulos:

- **Módulos orquestadores**: Administran la operación autónoma (*Planning*, *Supervisor* y *Executor*).
- **Módulos especializados**: Proporcionan capacidades específicas (*Knowledge*, *HRI*, *Vision*, *Navigation*, *Manipulation* y *Hardware*).

Estos módulos se ejecutan en dos computadoras comunicadas vía Ethernet, formando un único grafo de ROS 2. La computadora portátil, montada en un soporte dedicado detrás del torso y sobre la base móvil, concentra prácticamente todos los paquetes; la computadora embebida, en cambio, únicamente alberga algunos paquetes de control de *Hardware* y la interfaz de *HRI*. La estructura de cada módulo se documenta a continuación.

A.1. Planning

El módulo *Planning* traduce comandos en lenguaje natural o metas estructuradas en un plan ejecutable de instrucciones. El procesamiento inicia en el paquete `semantic_parser` y concluye en `clips_node`, que publica la secuencia de acciones en `/planning/clips/results` para el módulo *Supervisor*.

A.1.1. Paquete `planning_tools`

Proporciona la biblioteca `Planning` que encapsula en llamadas síncronas los servicios del módulo bajo el *namespace* `/planning`. Aloja el *launch* raíz `planning.launch.xml`.

A.1.2. Paquete `clips_node`

Envuelve el motor de reglas CLIPS mediante la biblioteca `clips` y ofrece las utilidades de conversión de metas.

1. **clips_node**:
   Carga las reglas indicadas en el parámetro `plan_name`, que corresponden a categorías de RoboCup@Home como `gpsr_planning` y `restaurant_planning`. Realiza la inferencia en un hilo independiente para no bloquear el *executor* de ROS 2.

2. **goals_to_clips_node**:
   Convierte una lista JSON de metas `goal(args)` en hechos CLIPS por lote antes de disparar la planeación.

A.1.3. Paquete `semantic_parser`

Interpreta comandos en lenguaje natural.

1. **qwen_semantic_node**:
   Consulta un servidor HTTP externo basado en Qwen para transformar comandos transcritos en metas estructuradas.

2. **semantic_node**:
   Analiza oraciones con un *pipeline* de spaCy `en_core_web_trf` y genera representaciones semánticas estructuradas.

3. **entity_extractor_node**:
   Extrae entidades nombradas consultando los diccionarios del paquete `knowledge_bank`.

A.1.4. Paquete `clips_launch`

Contiene `clips.launch.xml`, que inicia `clips_node` y `goals_to_clips_node`.

A.2. Supervisor

El módulo *Supervisor* cierra el ciclo entre *Planning* y *Executor*. Convierte el plan serializado que produce CLIPS en una cola de instrucciones que el *Executor* solicita secuencialmente, valida y clasifica el resultado de cada tarea reportada y orquesta los comportamientos correctivos entre ambos módulos.

A.2.1. Paquete `supervisor_plan`

Recibe el plan serializado publicado en `/planning/clips/results`, lo divide en una cola interna de instrucciones secuenciales y las expone bajo demanda. Permite superponer instrucciones correctivas sin perder la cola original y rebobinar pasos. Sin un plan cargado, entrega la instrucción de respaldo `Justina listen keyword`.

1. **supervisor_plan_node**:
   Mantiene la cola de instrucciones del plan activo y gestiona su despacho bajo demanda.

A.2.2. Paquete `supervisor_execution`

Valida y clasifica el resultado de cada tarea que `executor_manager` reporta tras su ejecución en `executor_worker`. Incluye una clase validadora por cada acción del *Executor*, como `GoTo`, `Say`, `Listen`, `Take` y `Drop`, que normaliza el resultado crudo de las herramientas en un `TaskExecutionReport`.

1. **supervisor_execution_node**:
   Autoriza y clasifica la ejecución de tareas delegando en el validador de la acción correspondiente, y emite el pulso de disparo hacia el *Executor*.

A.2.3. Paquete `supervisor_manager`

Coordina el avance del plan y las acciones correctivas. Incorpora la política de recuperación `PnpRecoveryManager` para las pruebas de *Pick and Place*, con reintentos locales, verificación de agarre con `verify_take`, posposición de objetos fallidos y reordenamiento de tareas, y mantiene clientes directos hacia *HRI*, *Navigation* y *Hardware* para ejecutar acciones correctivas independientes del *Executor*. No forma parte de ningún *launch*: se ejecuta manualmente en terminal después de levantar `surge_et_ambula` o se lanza desde `gui_controller`.

1. **supervisor_manager_node**:
   Publica en `/supervisor/supervisor_manager/allow_next_instruction` la señal que genera el pulso hacia `/input_trigger` y permite avanzar entre las instrucciones del plan.

A.2.4. Paquete `supervisor_system`

Publica el estado operativo de los subsistemas del robot.

1. **supervisor_system_node**:
   Publica cada segundo un estado saludable fijo para todos los módulos.

A.2.5. Paquete `surge_et_ambula`

Centraliza el arranque global del robot con los *launch* raíz `justina.launch.xml` para el robot real, `justina_sim.launch.xml` para simulación y `map.launch.xml` para mapas, además de configuraciones alternativas de hardware.

A.3. Executor

El módulo *Executor* es la capa intermedia que ejecuta secuencialmente las instrucciones generadas por *Planning* y *Supervisor*. Desacopla el razonamiento deliberativo de alto nivel en CLIPS de la ejecución física de las primitivas de acción: navegación, visión, manipulación y HRI. Se divide en un despachador reactivo, `executor_manager`, que gestiona las solicitudes de instrucciones y su autorización, y un servidor de ejecución, `executor_worker`, que delega el movimiento físico a máquinas de estados finitos modularizadas en `executor_tools`.

A.3.1. Paquete `executor_tools`

Proporciona las máquinas de estados finitos del *namespace* `/executor` que encapsulan el comportamiento reactivo de Justina. Cada herramienta reporta retroalimentación periódica y devuelve al `executor_worker` un `Result` uniforme con `success`, `status` y `message`. Las primitivas disponibles son:

- **`Take` y `VisualGraspApproach`**: Ejecuta la prensión cinemática de objetos en mesas o repisas con trayectorias planificadas en MoveIt2. Genera candidatos de aproximación frontal o cenital, reordena la secuencia articular hombro/codo → muñeca → torso, corrige en lazo cerrado por servovisual con la base móvil y repliega el brazo invirtiendo analíticamente la trayectoria.

- **`VerifyTake`**: Inspecciona visualmente la escena tras la manipulación con OWLv2. Compara la escena actual con la pose previa registrada en `Knowledge`; si el objeto aún se detecta en la superficie, libera la asignación de la mano del robot y emite un fallo de prensión.

- **`Drop`**: Coloca objetos en mesas, estantes, contenedores o en el suelo mediante `floor`. Consume la pose calculada por `FindSpace` o `Knowledge`, planifica la aproximación con orientación fija de la herramienta $q_y = -0.707$ y $q_w = 0.707$, abre el *gripper* y retrae el brazo respetando prismas de exclusión para no colisionar con el objeto liberado.

- **`FindSpace`**: Segmenta la nube de puntos RGB-D con servicios dedicados para mesa, contenedor o estante. Valida los límites volumétricos en `base_link`, descarta transformadas obsoletas y almacena la entidad `safe_drop/1` con su pose global en `map` y su meta de aproximación relativa.

- **`Give`**: Transfiere un objeto a una persona. Identifica qué mano sostiene el artículo solicitado, extiende el brazo hacia adelante, anuncia por voz una cuenta regresiva, abre el efector final, espera $2s$ para el relevo manual y actualiza la vacancia de la mano en `Knowledge`.

- **`Receive`**: Recibe un objeto de un usuario. Verifica qué brazo está disponible, con prioridad para el izquierdo, extiende la extremidad con el *gripper* abierto, solicita verbalmente la entrega, cierra la pinza tras una pausa y actualiza la posesión del objeto en el modelo del mundo.

- **`FindObject`**: Barre con la cabeza para localizar objetos o mobiliario. Atiende consultas de asientos libres con `seat` y de detección abierta con OWLv2, centra el eje óptico en el blanco con `hardware_->LookAt()` y actualiza las coordenadas de manipulación 3D y de navegación 2D.

- **`FindPerson`**: Barre visualmente para localizar y seguir el rostro de personas. Estabiliza los servos del cuello con `waitForHeadStable`, centra el rostro en el encuadre con control en lazo cerrado y usa el reconocedor facial para identificar usuarios conocidos o registrar invitados nuevos como `guest_<id>`.

- **`FindPose`**: Detecta posturas humanas como sentado, apuntando o con la mano alzada. Proyecta una pose de aproximación escalada al 95 % para no invadir el espacio personal y registra la coordenada en el búfer compartido `customer_pose_map_` en el marco `map`.

- **`Align`**: Orienta la base paralelamente al borde de mesas o de cara a contenedores con un controlador proporcional angular a $20Hz$, $k_p = 0.025$ y banda de $[0.04, 0.20] rad/s$. Exige 3 muestras consecutivas con error menor a $1.5°$.

- **`Focus`**: Activa o desactiva el seguimiento y la fijación visual continua en el subsistema de visión mediante banderas booleanas.

- **`AnalyzeObjects`**: Aplica razonamiento visual sobre objetos en mesas: conteo acumulado con `count`, comparación cardinal entre clases con `compare` y filtrado por tamaño con `size` o por color con `color`. Registra la pose de la instancia elegida para su prensión inmediata con `Take`.

- **`AnalyzePerson`**: Inspecciona personas en la escena evaluando el color de la vestimenta, gestos dinámicos como `waving`, `pointing` y `raising`, o ambos, y genera una pose de aproximación hacia la persona analizada.

- **`Say`**: Orquesta la síntesis de voz extrayendo frases parametrizadas de `Knowledge`, resolviendo variables temporales como `day_of_week` o instrucciones de guiado, sustituyendo guiones bajos por pausas naturales y aceptando texto plano como respaldo.

- **`Listen`**: Captura comandos de voz en modo simple, con palabra clave directa, o en modo completo con el formato `<categoría>_<atributo>_<índice>`. Extrae entidades semánticas con hasta 3 reintentos y las registra en `Knowledge`.
- **`Point`**: Rota la base móvil para absorber el ángulo de paneo de la cabeza, suaviza el cuello a cero con filtrado exponencial $\alpha = 0.1$ y extiende el brazo izquierdo para señalar un objetivo.
- **`Check`**: Verifica condiciones físicas discretas del entorno con sensores de hardware, principalmente el estado abierto o cerrado de puertas mediante `CheckDoorState`.
- **`Follow`**: Alterna el servicio de rastreo de personas en lazo cerrado gestionado por la capa de HRI.
- **`GoTo`**: Desplaza la base hacia coordenadas topológicas predefinidas del mapa global mediante la navegación detallada.
- **`Approach`**: Aproxima el robot semánticamente a personas u objetos. Proyecta poses locales en `base_link` a globales en `map`, frena el controlador cinemático al entrar en tolerancia por TF, aplica corrección residual para colocación con `safe_drop` y desplaza la base lateralmente $0.15m$ para alinear el hombro que ejecutará la acción. En modo parcial mantiene un margen de $0.75m$ frente al usuario.
- **`SaveLoc`**: Registra ubicaciones topológicas en el gestor de mapas e intercepta casos especiales: puntos de retorno a clientes, avances con desfase angular de $+90°$ en `kitchen_bar` y poses de calibración inicial para el filtro AMCL.
- **`Fold`**: Ejecuta una coreografía bimanual para doblar prendas sobre mesas. Identifica puntos clave visuales, repliega secuencialmente ambas mangas hacia el interior, toma el dobladillo inferior y avanza la base $0.15m$ a $0.08m/s$ para depositar la mitad inferior sobre la prenda.

A.3.2. Paquete `executor_manager`

Orquesta la comunicación entre el *Supervisor* y `executor_worker` mediante un lazo asíncrono regulado por pulsos.

1. **executor_manager_node**:
   Solicita la instrucción activa al servicio `/request_instruction` al recibir un pulso en `/input_trigger`, la divide en agente, acción en minúsculas y argumentos para formar un `TaskStep`, y la despacha como meta de la acción `ExecuteTask` al servidor `/executor/execute_task`. Si el servicio responde `blocked_by_manager`, reintenta la solicitud cada $200ms$ con un *wall timer*. Al terminar la tarea, sea `SUCCESS` o `FAILED`, envía la acción, su argumento y el mensaje del *worker* al servicio `/supervisor/supervisor_executor/authorize` y espera el siguiente pulso del *Supervisor*.

A.3.3. Paquete `executor_worker`

Expone el servidor de acciones `/executor/execute_task` y despacha cada meta a la herramienta de `executor_tools` correspondiente mediante la tabla `actionMap_`.

1. **executor_worker_node**:
   Acepta toda meta entrante y responde con un `TaskResult::FAILURE` detallado cuando la acción es desconocida o malformada, de modo que el *Supervisor* pueda diagnosticar la causa. Ejecuta cada máquina de estados en un hilo desacoplado para mantener libre el *executor* de ROS 2 y reporta como `FAILED` las excepciones de las herramientas sin detener el proceso.

A.3.4. Paquete `gui_controller`

Ofrece una aplicación de escritorio PyQt5 para monitorear y depurar durante el desarrollo, con la que se lanzan, supervisan y detienen procesos del sistema.

1. **gui_controller**:
   Inspecciona el grafo del sistema con las APIs de `ros2node` y finaliza procesos mediante señales del sistema con `pkill`.

A.4. Knowledge

El módulo *Knowledge* mantiene y expone el estado del mundo del robot en tiempo de ejecución: personas, objetos, ubicaciones conocidas, atributos del robot y analítica de percepción. Provee además un banco de frases parametrizables que el módulo *HRI* utiliza para generar diálogo, resolviendo sus referencias contra el estado del mundo actualizado.

A.4.1. Paquete `knowledge_tools`

Proporciona la biblioteca `Knowledge` que encapsula en llamadas síncronas los servicios de `world_state_node` y `phrase_manager_node` bajo el *namespace* `/knowledge`. Permite a otros módulos consultar y actualizar el estado del mundo o solicitar frases resueltas.

A.4.2. Paquete `knowledge_bank`

Concentra la configuración estática y el estado persistente del módulo:

- **Dominios estáticos**: `domains/locations.yaml`, `objects.yaml` y `people.yaml`.
- **Banco de frases**: Archivos por contexto de prueba en `phrases/`: `gpsr.yaml`, `hri.yaml`, `inspection.yaml`, `laundry.yaml`, `pnp.yaml` y `restaurant.yaml`.

- **Vocabulario**: Diccionarios de bebidas, nombres, objetos y órdenes en `dictionaries/`.
- **Estado en ejecución**: `runtime/world.yaml`, actualizado y persistido de forma continua.

A.4.3. Paquete `knowledge_manager`

Gestiona activamente la información del módulo y la resolución de diálogos.

1. **world_state_node**:
   Mantiene en memoria el estado del mundo: robot, ubicaciones, objetos, personas y analítica. Se inicializa con los dominios de `knowledge_bank` y persiste cada cambio en `runtime/world.yaml`. Atiende las peticiones en grupos de *callbacks* independientes protegidos por un *mutex* de lectura y escritura que permite lecturas simultáneas.

2. **phrase_manager_node**:
   Carga las plantillas de diálogo de `knowledge_bank/phrases`. Selecciona una frase aleatoria para la clave solicitada y resuelve sus comodines con formato `{categoria:atributo:id}` o `categoria_atributo[_id]` consultando el estado del mundo.

A.5. HRI

El módulo HRI es responsable de establecer y mantener la interacción directa entre Justina y una persona específica, mediante comunicación verbal y seguimiento de su posición. Utiliza Whisper para transcripción de voz y Piper para síntesis de voz, y combina las detecciones de YOLO con la nube de puntos de la cámara y el LiDAR de la base para detectar, localizar y seguir a la persona objetivo, ejecutando los desplazamientos mediante el sistema de navegación.

A.5.1. Paquete `hri_tools`

Proporciona la biblioteca `HRI` que encapsula en llamadas síncronas los servicios de voz, seguimiento y detección de poses bajo el *namespace* `/hri`. Aloja el *launch* `hri.launch.xml`, que levanta los nodos `Talker`, `Listener`, `human_detector` y `human_follower`.

A.5.2. Paquete `human_detection`

Detecta personas, estima su posición y reconoce movimientos del cuerpo mediante visión.

1. **person_tracker**:
   Combina las detecciones de YOLO con la información 3D de la cámara para calcular la posición de la persona objetivo. Con estos datos Justina mantiene localizada a la persona, orienta la cabeza hacia ella y dispone de información para su seguimiento.

2. **pose_detector_node**:
   Detecta la postura de una persona, como saludar o apuntar con la mano. Se inicia desde `vision.launch.xml` y no desde `hri.launch.xml`.

A.5.3. Paquete `human_follower`

Sigue a una persona mientras Justina se desplaza, combinando la detección de piernas por LiDAR con la posición del cuerpo que entrega el sistema de visión.

1. **leg_finder_node**:
   Procesa la información del LiDAR de la base para identificar patrones que corresponden a piernas humanas.

2. **human_follower_nodeV2**:
   Estima la ubicación de la persona combinando la posición de las piernas de `leg_finder_node` con la del cuerpo de `person_tracker`, y genera los comandos para que el robot la siga a una distancia determinada.

3. **human_tracking_node**:
   Coordina el seguimiento. Al activarse, habilita la visión continua, la detección de piernas y `human_follower_nodeV2`. Supervisa además la confianza del seguimiento y lo desactiva si deja de detectar correctamente a la persona, para evitar que el robot siga una posición incorrecta.

A.5.4. Paquete `listener`

Reconoce la voz del usuario a través del micrófono del robot.

1. **listener_node**:
   Captura el audio del micrófono, detecta cuándo habla una persona y la transcribe con Whisper. Limita las respuestas esperadas con archivos de gramática JSON según la prueba.

A.5.5. Paquete `talker`

Convierte texto en audio para que Justina pueda hablar.

1. **talker_node**:
   Recibe el texto que debe decir el robot, lo convierte en audio con Piper y lo reproduce por las bocinas.

A.5.6. Paquete `justina_gui`

Ofrece un panel de operación en interfaz gráfica PyQt que se ejecuta en la computadora de la base Robotino 4 y se lanza desde `justina_hardware` bajo el *namespace* `/hardware`.

1. **justina_gui_node**:
   Muestra en pantalla el estado operativo del robot durante las pruebas: paro de emergencia, texto actual de síntesis y reconocimiento, e imágenes de depuración de visión.

A.6. Vision

El módulo *Vision* implementa la capa de percepción visual del robot mediante nodos de ROS 2 bajo el *namespace* `/vision`. Detecta objetos con YOLO y con OWLv2 para vocabulario abierto, reconoce rostros, realiza analítica de imágenes y convierte coordenadas 2D a 3D.

A.6.1. Paquete `vision_tools`

Proporciona la biblioteca `Vision` que encapsula en llamadas síncronas los servicios de detección, analítica y reconocimiento facial bajo el *namespace* `/vision`. Aloja el *launch* `vision.launch.xml`, que configura los parámetros de `vacancy_detector`.

1. **detect_seats_node**:
   Combina la detección de YOLO con la conversión a pose 3D para determinar si hay un asiento libre frente al robot.

A.6.2. Paquete `yolo_detect`

Detecta objetos con un modelo de visión por computadora.

1. **yolo_detect_node**:
   Ejecuta YOLO11s sobre la imagen de entrada para publicar de forma continua la lista de objetos detectados y responder a peticiones bajo demanda.

A.6.3. Paquete `owlv2_detect`

Detecta y analiza objetos de vocabulario abierto con OWLv2.

1. **OWL_service**:
   Ejecuta OWLv2 con ViT para detectar objetos a partir de consultas de texto.

2. **Analyze_node**:
   Realiza analítica sobre las detecciones de OWLv2: conteo con seguimiento IoU, filtrado por tamaño y color, y comparación. Solicita las detecciones a OWLv2 en cada operación válida. El `class_count` devuelto corresponde a los objetos de la consulta actual, y el nodo conserva además un historial interno de IDs.

A.6.4. Paquete `pc_bbox_pose`

Convierte detecciones 2D en poses tridimensionales.

1. **pc_bbox_pose_node**:
   Estima la pose tridimensional de un objeto a partir de un *bounding box* 2D y la nube de puntos 3D.

A.6.5. Paquete `face_recog`

Reconoce rostros y registra personas.

1. **face_recogFlorence2**:
   Implementa con Florence-2 el pipeline completo de percepción facial: detección, identificación, descripción y registro de personas.

A.6.6. Paquete `table_alignment`

Estima la geometría de superficies planas para la aproximación del robot.

1. **table_segmenter_node**:
   Segmenta la nube de puntos para encontrar el plano de una mesa, redonda o rectangular, y calcula el ángulo de alineación con su borde.

A.6.7. Paquete `vacancy_detector`

Localiza zonas despejadas para manipulación y depósito sobre mesas, repisas o contenedores.

1. **vacancy_cloud_filter_node**:
   Recorta la nube de puntos con un filtro ROI compartido y suspende temporalmente los filtros de navegación mientras la detección de espacio está activa.

2. **table_drop_tf_node**:
   Calcula puntos libres para colocar objetos en mesas y publica las transformaciones TF `table_frame` y `table_drop_start`.

3. **shelf_drop_tf_node**:
   Calcula puntos libres para colocar objetos en estantes mediante la detección de planos horizontales y publica las transformaciones TF `shelf_frame` y `shelf_drop_start`.

4. **container_drop_tf_node**:
   Calcula puntos libres para colocar objetos en contenedores acotando la región con OWLv2 y publica las transformaciones TF `container_frame` y `container_drop_start`.

A.7. Navigation

El módulo *Navigation* localiza al robot, mantiene un mapa de obstáculos dinámicos superpuesto al mapa estático, planea rutas sobre él y las ejecuta con evitación reactiva de obstáculos. `nav_controller_node` y `locations_manager_node` constituyen la interfaz pública del módulo y se ejecutan en el *namespace* raíz. Por debajo, `motion_planner_node` orquesta la maniobra hacia una meta con `path_planner_node` para la ruta, `simple_move_node` para el control de velocidad y `potential_fields_node` para la evitación reactiva. `map_augmenter_node` y `obstacle_detector_node` mantienen el mapa aumentado a partir del mapa estático de `nav2_map_server`, mientras que `nav2_amcl` o `slam_toolbox` localizan al robot.

A.7.1. Paquete `navigation_tools`

Proporciona la biblioteca `Navigation` que encapsula en llamadas síncronas los servicios de navegación a ubicaciones conocidas y de registro de la pose actual como ubicación nueva, expuestos en el *namespace* raíz.

A.7.2. Paquete `locations_manager`

Gestiona las ubicaciones nombradas del robot.

1. **locations_manager_node**:
   Guarda y consulta las ubicaciones nombradas de `config/known_locations.yaml` y difunde periódicamente su transformación TF para visualización.

A.7.3. Paquete `nav_controller`

Expone la interfaz de navegación a ubicaciones conocidas.

1. **nav_controller_node**:
   Traduce las instrucciones de ir a una ubicación conocida en una meta de pose concreta, calcula un *timeout* proporcional a la distancia por recorrer y difunde la señal de paro de navegación que consumen los *drivers* de hardware.

A.7.4. Paquete `navigation_start`

Reúne la configuración, los mapas y los *launch* de la pila de navegación: `navigation_justina_amcl.launch.xml`, que opera en modo AMCL por defecto, y `navigation_justina_slam.launch.xml`, que opera en modo SLAM mediante `slam_toolbox`. Incluye los mapas estáticos y de prohibición por escenario, entre ellos RoboCup 2026 y TMR 2026.

A.7.5. Paquete `mvn_pln`

Orquesta las maniobras de navegación completas.

1. **motion_planner_node**:
   Orquesta la maniobra hacia una meta. Solicita la ruta a `path_planner_node`, la despacha a `simple_move_node`, coordina `potential_fields_node` y reacciona a paros y eventos de riesgo de colisión.

A.7.6. Paquete `path_planner`

Planea rutas sobre el mapa de navegación.

1. **path_planner_node**:
   Busca rutas punto a punto con el algoritmo A*, con movimientos diagonales opcionales, sobre el mapa estático o el mapa aumentado.

A.7.7. Paquete `simple_move`

Sigue trayectorias y controla la velocidad de la base.

1. **simple_move_node**:
   Convierte rutas u objetivos en comandos de velocidad mediante control proporcional lineal y angular, con orientación opcional de la cabeza hacia la dirección de avance.

A.7.8. Paquete `potential_fields`

Evita obstáculos de forma reactiva.

1. **potential_fields_node**:
   Calcula un campo de repulsión reactivo con datos del LiDAR y la nube de puntos, y lo superpone al `cmd_vel` de `simple_move_node` para evitar colisiones con obstáculos dinámicos no mapeados.

A.7.9. Paquete `map_augmenter`

Mantiene el mapa aumentado que utiliza la planeación de rutas.

1. **map_augmenter_node**:
   Combina el mapa estático, el mapa de prohibición y los obstáculos dinámicos en un mapa aumentado y en su representación de costos.

2. **obstacle_detector_node**:
   Detecta obstáculos dinámicos en el LiDAR y la nube de puntos dentro de una ventana espacial configurable y publica un `OccupancyGrid` con decaimiento temporal.

A.7.10. Paquete `augment_gridmap_online`

Edita de forma interactiva el mapa de prohibición.

1. **augment_gridmap_online_node**:
   Añade obstáculos permanentes al mapa de prohibición mediante clics de selección en RViz.

A.7.11. Paquete `mapless_nav`

Implementa navegación reactiva sin mapa global mediante campos potenciales, un árbol de comportamiento y un modelo recurrente entrenado. Se lanza desde `justina_mapless_wbt.launch.xml` en el entorno de simulación Webots.

A.7.12. Paquete `debugger`

Proporciona la biblioteca cliente `DebugClient` para enviar mensajes de estado desde cualquier nodo hacia una consola unificada en terminal.

1. **debug_manager.py**:
   Despliega en una única consola los mensajes de estado enviados con `DebugClient`.

A.8. Manipulation

El módulo *Manipulation* es responsable de planear y coordinar los movimientos de los brazos y el torso para la manipulación de objetos a partir de una pose objetivo. Utiliza MoveIt2 con TRAC-IK para resolver la cinemática inversa y el algoritmo RRT-Connect sobre un OctoMap del entorno para generar trayectorias libres de colisión, ejecutándolas mediante los servicios de bajo nivel del módulo *Hardware*.

A.8.1. Paquete `manipulation_tools`

Proporciona la biblioteca `Manipulation` que encapsula en llamadas síncronas los servicios de planeación y seguimiento de trayectorias bajo el *namespace* `/manipulation`. Aloja el *launch* `manipulation.launch.xml`, que levanta los nodos `move_group`, `manipulation_planner_node`, `manipulation_controller_node` y `manipulation_cloud_filter_node`, y remapea `/manipulation/joint_states` a `/hardware/joint_states`.

1. **manipulation_cloud_filter_node**:
   Recorta la nube de puntos de la cámara a una región de interés y reduce su densidad por *voxelización*. Publica el resultado para alimentar el OctoMap de MoveIt2 y se habilita bajo demanda.

A.8.2. Paquete `manipulation_planner`

Expone el servicio de planeación cinemática encapsulando MoveIt2.

1. **manipulation_planner_node**:
   Recibe una pose objetivo para el efector final y determina el tipo de agarre, *pinch* o envolvente, según el *tip link* especificado. Solicita la trayectoria a MoveIt2 mediante RRTConnect a partir del estado publicado en `/hardware/joint_states` y, opcionalmente, coordina el filtrado de la nube de puntos, la limpieza del OctoMap y la exclusión del objeto objetivo como obstáculo.

A.8.3. Paquete `manipulation_controller`

Expone el servicio de control de trayectorias utilizando los servicios de *Hardware*.

1. **manipulation_controller_node**:
   Recibe una trayectoria y la descompone en *waypoints* integrados por las posiciones del torso y de las seis articulaciones del brazo correspondiente. Envía los *waypoints* en paralelo a los nodos de hardware y coordina la ejecución bloque por bloque.

A.8.4. Paquete `move_it`

Contiene la configuración cinemática y semántica de Justina, que incluye las definiciones URDF y SRDF, los grupos de planeación y los parámetros de TRAC-IK requeridos por el nodo `move_group` de MoveIt2.

A.8.5. Paquete `trac_ik`

Integra la biblioteca de TRACLabs Robotics como *plugin* de cinemática inversa para MoveIt2.

A.9. Hardware

El módulo *Hardware* abstrae los dispositivos conectados al robot y expone interfaces bajo el *namespace* `/hardware` para que otros módulos reciban información del sistema o envíen comandos de movimiento. El hardware del robot se compone de cabeza, torso, brazos, pinzas, base omnidireccional, cámara RGB-D y LiDAR. Los paquetes de este módulo se ejecutan en la computadora embebida de la base, salvo indicación contraria.

A.9.1. Paquete `hardware_tools`

Proporciona la biblioteca `Hardware` que encapsula en llamadas síncronas los servicios de movimiento y consulta de los dispositivos del robot bajo el *namespace* `/hardware`. Aloja el *launch* `hardware.launch.xml`, que inicia los nodos del módulo que se ejecutan en la computadora principal. Reside en la computadora principal para que los módulos orquestadores puedan acceder a ella.

A.9.2. Paquete `justina_description`

Proporciona el modelo estructural del robot en un archivo URDF, con la jerarquía de eslabones y sus relaciones mediante diferentes tipos de juntas, para visualizar su estado durante la ejecución. Se duplica byte a byte en la computadora principal.

1. **simple_justina_sim**:
   Reemplaza a los *drivers* reales en una simulación ligera. Integra la velocidad comandada, publica la odometría, la TF de `odom` a `base_link`, articulaciones falsas y un láser simulado, e implementa directamente los servicios de movimiento de cabeza, torso y brazos.

2. **kinect_tilt_motor_node**:
   Refleja el *tilt* de cabeza comandado en el motor físico de inclinación del Kinect v1 por USB. Se lanza únicamente cuando la variable `use_kinect` está activa.

3. **depth_to_pointcloud**:
   Convierte una imagen de profundidad y su `camera_info` en una nube de puntos `sensor_msgs/PointCloud2` para el entorno de simulación Webots.

A.9.3. Paquete `door_detector`

Evalúa el estado de la puerta frente al robot. Se ejecuta en la computadora principal.

1. **door_detector_node**:
   Determina si una puerta está abierta o cerrada comparando la distancia promedio del sector central del barrido láser de la base con un umbral de $0.75m$.

A.9.4. Paquete `scanner`

Fija la cabeza en dirección al punto de interés establecido por el módulo de visión. Se ejecuta en la computadora principal.

1. **head_control_node**:
   Publica la posición de la cabeza a partir de un punto 3D de interés recibido en `/hardware/head/look_at`. Calcula pan y tilt mediante TF entre el marco de la cámara o el cuello y el objetivo, respeta los límites articulares y aplica una zona muerta para evitar micromovimientos. Evita además mover la cabeza durante las fases críticas de navegación.

A.9.5. Paquete `justina_hardware`

Realiza el *bringup* de la base Robotino. Bajo el *namespace* `/hardware`, remapea los tópicos `joint_states` relativos de cada *driver* al tópico canónico `/hardware/joint_states`, incluye los *launch* de los subcomponentes, ejecuta la GUI del panel de operación y levanta un `robot_state_publisher` local con entrada remapeada a `/manipulation/joint_states`.

A.9.6. Paquete `base`

Reúne los paquetes y *drivers* de *Festo* con implementación en ROS 2 necesarios para controlar la base móvil Robotino.

1. **base_node**:
   Controla la base Robotino 4 conectándose por TCP/IP al servidor *RobotinoAPI2* del fabricante a través de la clase `ComROS`. Expone la tracción omnidireccional, los *bumpers*, las lecturas de motores, la gestión de energía, la IMU, las entradas y salidas digitales y los *encoders*.

2. **robotino_odometry_node**:
   Calcula y publica la odometría de la base a partir de las lecturas del *driver* como ejecutable independiente del nodo principal. Publica la TF `odom` cuando `publish_odom_tf` está activo.

3. **digital_stop_node**:
   Mapea el estado de la entrada digital física `DI7`, conectada al botón de paro, a la señal de alto que consumen navegación, brazos, cabeza y pinzas.

A.9.7. Paquete `head`

Proporciona el *driver* para el control de posición y velocidad de la cabeza.

1. **head_node**:
   Controla los 2 servomotores Dynamixel de pan y tilt de la cabeza por RS-485 mediante el SDK de Dynamixel.

A.9.8. Paquete `torso`

Proporciona el *driver* para el control de posición del torso.

1. **torso_node**:
   Controla el eje lineal del torso con un controlador Jrk de Pololu conectado al puerto USB `/dev/torso`. Usa retroalimentación analógica de posición con tolerancia y verificación de lecturas estables.

A.9.9. Paquete `arms`

Proporciona los *drivers* para el control de posición y velocidad de los manipuladores del robot.

1. **arms_node**:
   Controla los 6 servomotores Dynamixel de cada brazo sobre un bus RS-485 e incluye un esquema de compensación de deformación. Se instancia dos veces, parametrizado por `left_arm.yaml` y `right_arm.yaml`.

A.9.10. Paquete `grippers`

Proporciona los *drivers* para el control de las pinzas del robot.

1. **grippers_node**:
   Controla las pinzas por TCP/IP hacia un microcontrolador RP2350 con un protocolo binario propio de 8 bytes. Se instancia dos veces, parametrizado por `left_gripper.yaml` y `right_gripper.yaml`.

A.9.11. Paquete `joint_states_parser`

Compila los estados de las juntas que proporciona cada *driver* para generar el tópico canónico `/hardware/joint_states`.

1. **joint_states_parser_node**:
   Fusiona las lecturas parciales de articulaciones de los *drivers* de base, torso, brazos, pinzas y cabeza, y genera un mensaje completo en orden canónico para MoveIt2.

A.9.12. Paquete `urg_node`

Provee las mediciones planares de distancia para el mapeo, la localización y la evasión de obstáculos. Se instala como dependencia externa de ROS 2.

1. **urg_node**:
   Lee el LiDAR 2D *Hokuyo* de la base por el puerto USB `/dev/lidar`, acotado a 180 grados, y publica la lectura procesada.

Apéndice B

Protocolo de investigación

El presente protocolo establece la estructura para la recopilación de datos orientada a la identificación de necesidades de un robot de servicio doméstico. El instrumento integra las guías de entrevista dirigidas a distintos perfiles de usuario, así como las consideraciones éticas, los criterios de selección de participantes y los lineamientos de confidencialidad.

Entrevista a usuario final

1. ¿En qué tareas del hogar le gustaría que un robot lo asista?

2. ¿Hay actividades que preferiría hacer usted mismo en lugar de delegarlas a un robot? ¿Por qué?

3. ¿Qué tanto espera que el robot trabaje de forma autónoma en lugar de recibir órdenes explícitas?

4. ¿Cuánto tiempo cree que debería tomarle a un robot realizar sus tareas respecto al tiempo que le tomaría a una persona?

5. Si usted saliera de viaje y el robot se quedara solo en su casa, ¿qué le pediría que hiciera en su ausencia?

6. ¿De qué manera le gustaría poder dar órdenes al robot (escritas, habladas, desde un control, una computadora)?

7. ¿De qué forma se imagina que sería más fácil confirmar que el robot realmente entendió la órden que está por ejecutar?

8. ¿Le gustaría que el robot le reportara el estatus de las actividades encargadas? ¿En qué forma le gustaría que lo hiciera?

9. ¿Qué tan importante le parece que el robot parezca una persona?

10. ¿Cree que el robot debería cambiar su comportamiento dependiendo de con quién interactúa? En caso afirmativo, ¿de qué forma?

11. ¿Qué artículo del hogar le daría más miedo permitir que un robot manipule?

12. ¿Cuáles son los objetos más pesados o frágiles sería útil para usted que el robot pudiera manipular?

13. Además de objetos y bolsas, ¿qué otro tipo de cosas crees que sería útil que el robot pudiera manipular? Ej. Puertas, llaves, cepillos

14. ¿Le gustaría que el robot incluyera un manual para que usted mismo le pudiera dar mantenimiento y hacer reparaciones básicas o preferiría llevarlo con el proveedor directamente ante cualquier falla?

15. ¿Utilizaría el robot únicamente en su hogar o consideraría llevarlo consigo para que lo asista en viajes?

16. ¿La apariencia de qué robot le inspira mayor confianza? ¿Por qué?



(a) Opción 1



(b) Opción 2



(c) Opción 3

17. ¿Cuáles son las principales preocupaciones que le generaría tener un robot de servicio en casa?

18. ¿Qué elementos podría incluir el robot para hacerlo sentir más seguro?

Entrevista al desarrollador de software (General)

1. Cuéntanos acerca de tu experiencia en la RoboCup, enfocada principalmente a la categoría @Home.

2. ¿Cuáles han sido los problemas más importantes que has enfrentado o te has enterado que han enfrentado otros equipos durante una competencia?

3. ¿Cuál consideras que es el hardware mínimo que se requiere para participar en esta categoría? ¿Cuál de esos elementos consideras que tiene un mayor impacto en el desempeño del robot?

4. ¿Consideras que la apariencia del robot afecta la manera en la que el usuario interactúa con él? ¿Crees que este efecto tiene un impacto real en su desempeño?

5. Además de las instrucciones que se le dan a un usuario no experto antes de una prueba, ¿qué otros elementos integrados en el robot has notado que hacen más intuitiva la interacción?

6. ¿Qué otras formas de interacción se han usado cuando el usuario no domina el idioma inglés?

7. ¿Has identificado alguna característica del diseño de los manipuladores, que nosotros u otros equipos hayan implementado, que pudiera incomodar/lastimar al usuario?

8. ¿Qué tareas de manipulación consideras más críticas en la competencia?

9. ¿Cuál consideras una mejor característica al tomar objetos: la velocidad o la precisión? ¿Por qué?

10. ¿Cuál has identificado como el principal motivo por el que a un robot se le caen objetos que ya había tomado?

11. ¿Cuál es el objeto más grande o pesado que se debe tomar en las pruebas?

12. ¿Cuál es el objeto más pequeño y delicado que se ha tenido que manipular en las pruebas?

13. ¿Hay alguna otra característica, además del peso y tamaño, que haga complicado tomar un objeto? (Ej. Rugosidad de la superficie)

14. ¿Qué condiciones de luz has identificado que afectan más a los sistemas de percepción del robot?

15. ¿Has identificado problemas con puntos ciegos del robot en la competencia?

16. ¿Consideras que la medición de par o fuerza en las articulaciones del brazo y gripper sería útil?

17. ¿Qué medidas de redundancia crees que sean más importantes para la seguridad y el rendimiento del robot?

18. ¿Qué elementos de seguridad, además del botón de paro, consideras que es importante incluir en el diseño del robot?

Entrevista al desarrollador de software (Plataforma experimental)

1. ¿Cuáles consideras que son las características clave del manipulador para tomar objetos y cuáles pueden ser compensadas con la posición de la base?

2. ¿Cuál recuerdas como el efector final más efectivo en la competencia?

3. ¿Crees que la integración de una cámara en el gripper mejoraría su habilidad de tomar objetos?

4. ¿Hay alguna característica del robot que consideres importante como el peso o el tamaño? Más allá de las limitaciones que impone el reglamento.

5. En caso de contar con un robot de dos brazos, ¿consideras que deberían ser brazos idénticos o cada brazo debería tener una habilidad diferente? ¿El efector final debería ser intercambiable?

6. En los diseños previos, ¿qué partes del hardware eran más inaccesibles para su mantenimiento?

Entrevista al desarrollador de software (Plataforma comercial)

1. ¿Cuáles consideras que son las mayores virtudes del hardware de Takeshi?

2. Si pudieras modificar algo, ¿lo harías? ¿qué sería?

3. ¿Crees que la integración de la cámara en el gripper mejora su habilidad de tomar objetos o sólo incrementa el tiempo de procesamiento? ¿Sí la utilizan?

4. ¿Considerarías útil que el efector final fuera intercambiable?

5. ¿Qué tareas de manipulación consideras que son las más desafiantes considerando que Takeshi solo cuenta con un brazo?

6. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?

7. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?

8. ¿Existe alguna limitación en el mantenimiento del hardware considerando que se trata de un robot comercial?

9. Dado que todos tienen el mismo hardware, ¿qué utilizas como diferenciador?

Riesgos y beneficios anticipados

Riesgos

Esta investigación presenta riesgos mínimos para los participantes. No se utiliza instrumento alguno o contacto con el robot de ningún tipo y únicamente involucra preguntas sobre experiencias técnicas y necesidades cotidianas.

Los riesgos identificados incluyen:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.
- Fatiga durante la sesión de entrevista (mitigado mediante sesiones de duración razonable con pausas si es necesario).
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Beneficios

Los participantes podrán obtener los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Criterios de selección

Los participantes serán seleccionados mediante muestreo intencional conforme a dos perfiles específicos:

Perfil 1: Equipo de desarrollo

- Ser miembro activo o previo del equipo de desarrollo del robot de servicio doméstico Justina.
- Contar con experiencia de participación en la competencia RoboCup@HOME.
- Tener conocimiento técnico sobre el diseño, programación o implementación de funcionalidades del robot.

Perfil 2: Usuarios potenciales

- Ser personas que requieren asistencia doméstica constante debido a edad avanzada (65 años o más) o alguna condición de discapacidad que limite su autonomía en actividades cotidianas del hogar.
- Residir de forma independiente o semi-independiente en un entorno doméstico.
- Tener capacidad cognitiva para participar en una entrevista y proporcionar consentimiento informado (o contar con representante legal en caso necesario).

Privacidad y confidencialidad

Los datos personales recolectados durante la investigación serán utilizados exclusivamente con fines académicos. Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador.

Apéndice C

Carta de consentimiento informado

Por medio de la firma del presente consentimiento informado **autoriza de manera completamente voluntaria** el uso de toda la información proporcionada durante esta entrevista y encuestas de seguimiento, así como de su información de contacto, en el desarrollo del trabajo de tesis titulado **Rediseño de los manipuladores de un robot de servicio de propósito general** elaborada por Rafael Cuéllar Ramírez y Marco Elian Soriano Pimentel, quienes fungen como investigadores principales y serán las únicas dos personas con acceso a la información recabada. El trabajo se desarrolla en el Laboratorio de Bio-Robótica adscrito al Departamento de Procesamiento de Señales de la División de Ingeniería Eléctrica de la Facultad de Ingeniería de la UNAM, como único centro de investigación participante.

El trabajo de tesis tiene como objetivo el diseño de los manipuladores y efectores finales de un robot de servicio de propósito general, así como el algoritmo de control, para permitirle tomar objetos de uso doméstico.

Como parte de la metodología de diseño centrada en el usuario de Karl T. Ulrich y Steven D. Eppingerm, se requiere conocer la problemática que se pretende resolver a través de las experiencias de los usuarios potenciales y/o de aquellos que ya hayan utilizado tecnologías similares. Con este fin se realizará una entrevista inicial con una duración aproximada de una hora y encuestas de seguimiento posteriores. Durante la entrevista se grabará audio y se tomarán notas. Las encuestas de seguimiento se enviarán por medios electrónicos.

Su participación en este trabajo de investigación no representa riesgo de lesiones o problemas de salud relacionados, por lo tanto, no se contemplan normas de indemnización. Sin embargo, su participación conlleva los riesgos previsibles y no previsibles que se listan a continuación:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.
- Fatiga durante la sesión de entrevista.
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Por otro lado, aunque **no se contempla una remuneración económica**, su participación conlleva los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador. La información será utilizada con fines estrictamente académicos y de investigación y, una vez concluido el trabajo, todas las grabaciones serán eliminadas.

Al autorizar su participación, el equipo investigador se compromete a compartir con usted los resultados a través de los trabajos derivados que sean publicados.

En caso de que tenga alguna duda o inquietud relacionada con su participación, puede contactar a los investigadores principales. Adicionalmente, **puede retirarse libremente en cualquier momento sin consecuencia alguna**, mientras la investigación esté en curso mediante la **Carta de revocación de consentimiento** adjunta.

El equipo de investigación se reserva el derecho de terminar su participación sin su consentimiento en caso de que la información proporcionada no resulte relevante para el desarrollo del tema de investigación.

Este proyecto está avalado por el **Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería**.

Apéndice D

Necesidades identificadas

Este compendio organiza las necesidades identificadas en una estructura jerárquica de tres niveles, ordenados de forma descendente conforme al nivel de detalle, desde las necesidades más generales hasta las más específicas. Los 690 enunciados originales se sintetizaron en 122 necesidades estandarizadas, seleccionando en cada caso la redacción más representativa. Las Tablas D.1 a D.14 presentan las 14 necesidades primarias resultantes, donde la frecuencia de cada nivel corresponde al acumulado de las menciones que agrupa.

Tabla D.1. Necesidad primaria: *El robot manipula artículos domésticos.*

Necesidad	**Frec.**
El robot manipula artículos domésticos.	**245**
El robot manipula artículos domésticos.	**124**
El robot entrega artículos domésticos.	6
El robot manipula artículos ubicados por encima de su plano medio.	18
El robot manipula artículos domésticos.	15
El robot manipula artículos ubicados en zonas inaccesibles para el usuario.	14
El robot manipula artículos ubicados por debajo de su plano medio.	12
El robot manipula artículos con exactitud.	11
El robot manipula artículos pequeños.	10
El robot manipula artículos pesados.	9
El robot manipula artículos delgados.	7
El robot manipula artículos con formas diversas.	6
El robot manipula artículos grandes.	6

Continúa en la siguiente página

Tabla D.1. Necesidad primaria: *El robot manipula artículos domésticos.* (continuación).

Necesidad	**Frec.**
El robot manipula varios artículos de manera simultánea.	3
El robot manipula artículos en diferentes orientaciones.	3
El robot manipula artículos deformables.	2
El robot manipula artículos con superficies lisas.	2
El robot planea trayectorias del manipulador.	**70**
El robot dispone sus manipuladores en poses compatibles con otros sistemas.	13
El robot ajusta la estrategia de manipulación de los artículos conforme al entorno.	10
El robot ajusta de forma dinámica la trayectoria de sus manipuladores conforme a la posición de los artículos.	8
El robot ajusta la estrategia de manipulación conforme a las características de los artículos.	8
El robot ajusta la velocidad de manipulación para optimizar la ejecución de cada tarea.	6
El robot coordina sus manipuladores para realizar tareas colaborativas.	6
El robot sigue trayectorias suaves y controladas con sus manipuladores.	5
El robot permite definir y utilizar poses preestablecidas para sus manipuladores.	4
El robot planifica trayectorias óptimas para manipular artículos.	2
El robot evita trayectorias de riesgo para el usuario.	8
El robot extiende el alcance efectivo de sus manipuladores.	**21**
El robot extiende el alcance efectivo de sus manipuladores.	8
El robot dispone sus manipuladores en un amplio rango de poses.	5
El robot mantiene el equilibrio en todas sus poses.	8
El robot toma artículos con la fuerza necesaria.	**24**
El robot toma artículos con firmeza.	15
El robot toma artículos con suavidad.	9
El robot manipula equipo para realizar tareas domésticas.	**6**
El robot interactúa con puertas y cajones.	4
El robot manipula equipo para realizar tareas domésticas.	2

Tabla D.2. Necesidad primaria: *El robot atiende las órdenes del usuario.*

Necesidad	Frec.
El robot atiende las órdenes del usuario.	**75**
El robot facilita la interacción con el usuario.	**42**
El robot facilita la interacción con el usuario.	14
El robot complementa las indicaciones verbales con apoyos visuales.	8
El robot guía al usuario mediante indicaciones verbales.	8
El robot emite sonido.	1
El robot mantiene un estilo de comunicación uniforme.	1
El robot reporta el progreso y resultado de sus tareas.	4
El robot enfoca la atención del usuario.	5
El robot opera de forma silenciosa.	1
El robot atiende las órdenes del usuario.	**12**
El robot atiende las órdenes del usuario.	3
El robot interpreta comandos verbales.	4
El robot interpreta comandos visuales.	3
El robot interpreta comandos físicos.	2
El robot genera confianza en el usuario.	**20**
El robot genera confianza en el usuario.	7
El robot presenta características antropomórficas.	13
El robot reproduce música indicada por el usuario.	**1**
El robot reproduce música indicada por el usuario.	1

Tabla D.3. Necesidad primaria: *El robot realiza tareas domésticas.*

Necesidad	Frec.
El robot realiza tareas domésticas.	**37**
El robot limpia entornos domésticos.	**25**
El robot limpia entornos domésticos.	6
El robot ordena artículos domésticos fuera de lugar.	3
El robot cuida del jardín.	8
El robot lava utensilios domésticos.	4
El robot aspira pisos.	3
El robot talla muebles tapizados.	1
El robot realiza tareas domésticas.	**5**
El robot pinta muebles y paredes.	2
El robot cuida de las mascotas.	1
El robot lava ropa.	2
El robot prepara alimentos.	**6**
El robot sirve alimentos y bebidas.	4
El robot prepara alimentos.	2
El robot asiste al usuario mientras nada.	**1**
El robot asiste al usuario mientras nada.	1

Tabla D.4. Necesidad primaria: *El robot es robusto.*

Necesidad	Frec.
El robot es robusto.	**99**
El robot protege sus componentes.	**47**
El robot protege sus componentes contra agua.	12
El robot protege sus componentes contra impactos.	12
El robot protege sus conexiones eléctricas.	8
El robot protege sus conexiones mecánicas.	5
El robot protege sus componentes contra altas temperaturas.	2
El robot previene desconexiones mientras opera.	2

Continúa en la siguiente página

Tabla D.4. Necesidad primaria: *El robot es robusto.* (continuación).

Necesidad	Frec.
El robot soporta las cargas previstas.	6
El robot supervisa el estado de sus sistemas.	**52**
El robot identifica y reporta fallas en sus sistemas.	11
El robot interrumpe sus sistemas ante situaciones de riesgo.	10
El robot supervisa el estado de sus sistemas.	10
El robot restablece sus sistemas ante fallas.	6
El robot registra la tasa de éxito en la manipulación de artículos.	3
El robot verifica que los artículos hayan sido tomados con éxito.	12

Tabla D.5. Necesidad primaria: *El robot tiene un diseño intuitivo.*

Necesidad	Frec.
El robot tiene un diseño intuitivo.	**53**
El robot facilita su operación.	**25**
El robot facilita su operación.	8
El robot facilita el inicio de sus sistemas.	7
El robot integra sus sistemas de forma coherente.	4
El robot facilita su configuración.	2
El robot integra todos sus componentes de hardware.	2
El robot facilita el montaje de sus componentes.	1
El robot controla su hardware con el stack de software existente.	1
El robot facilita su mantenimiento.	**28**
El robot facilita su mantenimiento.	6
El robot mantiene sus componentes ordenados e identificados.	5
El robot sigue un esquema claro de diseño.	4
El robot facilita el desmontaje de sus componentes.	3
El robot opera sus componentes de forma independiente.	7
El robot incluye documentación técnica completa.	3

Tabla D.6. Necesidad primaria: *El robot identifica formas en el entorno.*

Necesidad	Frec.
El robot identifica formas en el entorno.	**50**
El robot identifica artículos en el entorno.	**44**
El robot identifica artículos en el entorno.	25
El robot determina la posición exacta de los artículos en el entorno.	8
El robot identifica artículos transparentes.	2
El robot construye modelos digitales de los artículos que identifica.	1
El robot distingue los artículos de la superficie que los soporta.	1
El robot identifica zonas de manipulación óptimas en los artículos.	3
El robot distingue superficies adyacentes.	1
El robot captura imágenes nítidas del entorno.	3
El robot identifica rostros en el entorno.	**6**
El robot identifica rostros en el entorno.	5
El robot advierte al usuario de visitantes desconocidos.	1

Tabla D.7. Necesidad primaria: *El robot es autónomo.*

Necesidad	Frec.
El robot es autónomo.	**19**
El robot planea tareas.	**12**
El robot modifica su plan de forma dinámica.	7
El robot genera un plan conforme a una meta establecida.	2
El robot agenda la ejecución de sus tareas.	2
El robot ayuda al usuario a organizar sus actividades.	1
El robot opera sin intervención del usuario.	**7**
El robot sigue un plan sin órdenes explícitas.	2
El robot determina su posición en el entorno sin intervención del usuario.	1
El robot brinda asistencia en emergencias domésticas.	4

Tabla D.8. Necesidad primaria: *El robot es portátil.*

Necesidad	Frec.
El robot es portátil.	**22**
El robot facilita su transporte.	**18**
El robot es ligero.	7
El robot facilita su transporte como equipaje personal.	6
El robot facilita el transporte individual de sus componentes.	3
El robot facilita su envío por servicios de paquetería.	2
El robot tiene dimensiones compatibles con entornos domésticos.	**4**
El robot tiene dimensiones compatibles con entornos domésticos.	3
El robot facilita su almacenamiento.	1

Tabla D.9. Necesidad primaria: *El robot navega en entornos domésticos.*

Necesidad	Frec.
El robot navega en entornos domésticos.	**35**
El robot navega en entornos domésticos.	**35**
El robot navega en entornos domésticos.	8
El robot navega evadiendo obstáculos.	15
El robot identifica obstáculos en el entorno.	8
El robot llega a su destino con exactitud.	4

Tabla D.10. Necesidad primaria: *El robot opera de forma continua por periodos prolongados.*

Necesidad	Frec.
El robot opera de forma continua por periodos prolongados.	**15**
El robot opera de forma continua por periodos prolongados.	**15**
El robot optimiza el consumo de energía.	5
El robot opera de forma continua por periodos prolongados.	9
El robot requiere poco mantenimiento.	1

Tabla D.11. Necesidad primaria: *El robot tiene sensores confiables.*

Necesidad	Frec.
El robot tiene sensores confiables.	**18**
El robot tiene sensores confiables.	**18**
El robot tiene sensores exactos y precisos.	1
El robot posiciona sus manipuladores con exactitud.	10
El robot opera bajo condiciones de iluminación variables.	7

Tabla D.12. Necesidad primaria: *El robot identifica la voz del usuario.*

Necesidad	Frec.
El robot identifica la voz del usuario.	**5**
El robot identifica la voz del usuario.	**5**
El robot registra sonido.	3
El robot identifica la voz del usuario.	2

Tabla D.13. Necesidad primaria: *El robot optimiza el tiempo de ejecución de sus tareas.*

Necesidad	Frec.
El robot optimiza el tiempo de ejecución de sus tareas.	**1**
El robot optimiza el tiempo de ejecución de sus tareas.	**1**
El robot optimiza el tiempo de ejecución de sus tareas.	1

Tabla D.14. Necesidad primaria: *El robot percibe profundidad.*

Necesidad	Frec.
El robot percibe profundidad.	**16**
El robot percibe profundidad.	**16**
El robot percibe profundidad.	16

Apéndice E

Necesidades seleccionadas

En este documento se detalla el procedimiento empleado para determinar la importancia relativa de las 14 necesidades primarias identificadas, así como el criterio de selección aplicado para orientar el diseño de los manipuladores de *Justina*. La importancia relativa combina la frecuencia relativa de los enunciados con la valoración promedio de una encuesta estructurada, distribuida electrónicamente a los tres usuarios entrevistados con el mayor volumen y diversidad de aportaciones, identificados aquí como Encuestado 1, 2 y 3.

La frecuencia relativa f_i se define como la razón entre los enunciados interpretados n_i correspondientes a la necesidad primaria i y la totalidad de los 690 registros. En la encuesta, cada participante calificó la relevancia de las necesidades primarias mediante una escala ordinal de cinco niveles:

- *Imprescindible*: 1.0
- *Deseable*: 0.6
- *Indiferente*: 0.4
- *Preferiría que no la tuviera*: 0.2
- *No debería tenerla*: 0.0

Con el promedio $\bar{p}_i$ de las tres ponderaciones $p_{i,k}$, la importancia relativa I_i se determina mediante la siguiente relación:

$$f_i = \frac{n_i}{690}, \qquad \bar{p}_i = \frac{1}{3}\sum_{k=1}^{3} p_{i,k}, \qquad I_i = f_i \cdot \bar{p}_i.$$

Dado el tamaño reducido de la muestra, los resultados de la encuesta se interpretan como indicadores cualitativos de tendencia, complementarios a los datos de frecuencia. Los valores se presentan redondeados, pero los cálculos se realizaron sin redondear.

La Tabla E.1 reúne las respuestas de cada encuestado, en el orden jerárquico de las necesidades primarias.

Tabla E.1. Respuestas de la encuesta para las 14 necesidades primarias

Necesidad primaria	**Encuestado 1**	**Encuestado 2**	**Encuestado 3**
El robot manipula artículos domésticos.	Imprescindible	Imprescindible	Deseable
El robot atiende las órdenes del usuario.	Imprescindible	Deseable	Imprescindible
El robot realiza tareas domésticas.	Deseable	Imprescindible	Deseable
El robot es robusto.	Deseable	Imprescindible	Imprescindible
El robot tiene un diseño intuitivo.	Deseable	Deseable	Deseable
El robot identifica formas en el entorno.	Deseable	Deseable	Deseable
El robot es autónomo.	Deseable	Imprescindible	Imprescindible
El robot es portátil.	Deseable	Indiferente	Deseable
El robot navega en entornos domésticos.	Imprescindible	Imprescindible	Deseable
El robot opera de forma continua por periodos prolongados.	Deseable	Deseable	Deseable
El robot tiene sensores confiables.	Deseable	Imprescindible	Imprescindible
El robot identifica la voz del usuario.	Indiferente	Deseable	Imprescindible
El robot optimiza el tiempo de ejecución de sus tareas.	Deseable	Deseable	Deseable
El robot percibe profundidad.	Indiferente	Imprescindible	Deseable

La Tabla E.2 condensa el conteo de enunciados, la frecuencia relativa, la valoración promedio y la importancia relativa calculada para cada necesidad primaria. Al aplicar un umbral de corte del 10 % ($I_i > 0.10$), sobresalen únicamente dos necesidades: *El robot manipula artículos domésticos*, con 0.308, y *El robot es robusto*, con 0.124.

Tabla E.2. Importancia relativa de las 14 necesidades primarias

Necesidad	Frec.	Frec. relativa	Promedio de evaluaciones	Importancia relativa
El robot manipula artículos domésticos.	**245**	**0.355**	**0.87**	**0.308**
El robot atiende las órdenes del usuario.	75	0.109	0.87	0.094
El robot realiza tareas domésticas.	37	0.054	0.73	0.039
El robot es robusto.	**99**	**0.143**	**0.87**	**0.124**
El robot tiene un diseño intuitivo.	53	0.077	0.60	0.046
El robot identifica formas en el entorno.	50	0.072	0.60	0.043
El robot es autónomo.	19	0.028	0.87	0.024
El robot es portátil.	22	0.032	0.53	0.017
El robot navega en entornos domésticos.	35	0.051	0.87	0.044
El robot opera de forma continua por periodos prolongados.	15	0.022	0.60	0.013
El robot tiene sensores confiables.	18	0.026	0.87	0.023
El robot identifica la voz del usuario.	5	0.007	0.67	0.005
El robot optimiza el tiempo de ejecución de sus tareas.	1	0.001	0.60	0.001
El robot percibe profundidad.	16	0.023	0.67	0.015
Total	**690**	**1.000**		

Al triplicar el umbral de corte, la necesidad *El robot manipula artículos domésticos* se desglosó en sus secundarias, mostradas en la Tabla E.3, heredando la calificación promedio de la necesidad primaria.

Tabla E.3. Necesidades secundarias de *El robot manipula artículos domésticos.*

Necesidad	Frec.	Frec. relativa	Promedio de evaluaciones	Importancia relativa
El robot manipula artículos domésticos.	124	0.180	0.87	0.156
El robot planea trayectorias del manipulador.	70	0.101	0.87	0.088
El robot extiende el alcance efectivo de sus manipuladores.	21	0.030	0.87	0.026
El robot toma artículos con la fuerza necesaria.	24	0.035	0.87	0.030
El robot manipula equipo para realizar tareas domésticas.	6	0.009	0.87	0.008
Total	**245**	**0.355**	**0.87**	**0.308**

Finalmente, la Tabla E.4 sintetiza el conjunto de necesidades a satisfacer mediante el diseño de los manipuladores. Este abarca las dos necesidades primarias seleccionadas junto con las cuatro secundarias que detallan aspectos específicos de la manipulación.

Tabla E.4. Necesidades seleccionadas para el diseño de los manipuladores

Necesidad	Importancia relativa
El robot manipula artículos domésticos.	**0.308**
El robot planea trayectorias del manipulador.	0.088
El robot toma artículos con la fuerza necesaria.	0.030
El robot extiende el alcance efectivo de sus manipuladores.	0.026
El robot manipula equipo para realizar tareas domésticas.	0.008
El robot es robusto.	**0.124**

Bibliografía

[1] B. Siciliano and O. Khatib, "Robotics and the handbook," in *Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), ch. 1, pp. 1–6, Springer, 2da ed., 2016.

[2] L. E. Sucar Succar, "Introducción," in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 1, pp. 1–5, Academia Mexicana de Computación, A.C., 2da ed., 2019.

[3] J. Savage Carmona, "Robots de servicio," in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 2, pp. 7–23, Academia Mexicana de Computación, A.C., 2da ed., 2019.

[4] T. Bajd and M. Mihelj, *Introduction to Robotics*. Springer, 1ra ed., 2013.

[5] R. C. Arkin, *Behavior-Based Robotics*. MIT Press, 1ra ed., 1998.

[6] B. Gates, "A robot in every home," in *Scientific American, Inc.*, pp. 58–65, 2006.

[7] J. Stückler and S. Behnke, "Integrating indoor mobility, object manipulation, and intuitive interaction for domestic service tasks," in *2009 9th IEEE-RAS International Conference on Humanoid Robots*, pp. 506–513, 2009.

[8] RoboCup Federation, "A brief history of RoboCup." `https://www.robocup.org/a_brief_history_of_robocup`, 2025. Accessed: 2025-10-06.

[9] RoboCup Federation, "RoboCup@Home League." `https://athome.robocup.org/`, 2025. Official website of the RoboCup@Home league for autonomous service robots.

[10] A. Bicchi and V. Kumar, "Robot grasping and contact: A review," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings*, pp. 348–353, 2000.

[11] M. Prats, P. J. Sanz, E. Martínez, R. Marín, and A. P. del Pobil, "Manipulación autónoma multipropósito en el robot de servicios jaume-2," in *Revista Iberoamericana de Automática e Informática Industrial*, pp. 25–37, 2008.

[12] BioRobotics Laboratory, UNAM, "Pumas OPL (Justina Robot)." [Online]. Available: https://biorobotics.fi-p.unam.mx/robot-justina/, 2025. Accessed: Oct. 1, 2025.

[13] C. Peralta Vázquez, "Justina, robot de servicio creado en la unam." https://www.uv.mx/prensa/ciencia/justina-robot-de-servicio-creado-en-la-unam/, 2019. Accessed: 2025-03-15.

[14] UNAM Global, "En 30 años, robots con inteligencia artificial, que atenderían a los humanos." https://unamglobal.unam.mx/global_revista/en-30-anos-creariamos-robots-con-inteligencia-artificial-que-atenderian-a-los-hum 2017. Accessed: 2026-08-19.

[15] J. Savage, A. Llarena, G. Carrera, S. Cuellar, D. Esparza, Y. Minami, and U. Peñuelas, *ViRbot: A System for the Operation of Mobile Robots*, vol. 5001, pp. 512–519. 05 2007.

[16] M. A. Negrete Villanueva, *Sistema de navegación para un robot de servicio*. Tesis de doctorado, Universidad Nacional Autónoma de México, Posgrado en Ciencia e Ingeniería de la Computación, Ciudad de México, México, 2019.

[17] Open Robotics, "About ROS."

[18] Open Robotics, "ROS 2 Concepts: Nodes." https://docs.ros.org/en/ros2_documentation/Concepts/Basic/About-Nodes.html. Accessed: 2026-08-11.

[19] B. Gerkey, "Why ROS 2?." https://design.ros2.org/articles/why_ros2.html, June 2014. Last modified: May 2022.

[20] Open Robotics, "ROS 2 Concepts: Topics, Services, and Actions."

[21] J. J. Craig, *Introduction to Robotics: Mechanics and Control*. Upper Saddle River, NJ: Pearson Prentice Hall, 3 ed., 2005.

[22] R. L. Norton, *Diseño de maquinaria: Síntesis y análisis de máquinas y mecanismos*. México, D.F.: McGraw-Hill Interamericana, 5 ed., 2013.

[23] F. Reuleaux, *The Kinematics of Machinery*. New York: Dover Publications, 1963.

[24] J. J. Uicker, G. R. Pennock, and J. E. Shigley, *Theory of Machines and Mechanisms*. New York: Oxford University Press, 3 ed., 2003.

[25] T. Kivelä, *Increasing the Automation Level of Serial Robotic Manipulators with Optimal Design and Collision-free Path Control*. Doctoral dissertation, Tampere University of Technology, Tampere, Finland, 2017.

[26] K. J. Waldron and J. Schmiedeler, "Kinematics," in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 11–36, Springer, 2 ed., 2016.

[27] Open Robotics, "Quaternion fundamentals." `https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Quaternion-Fundamentals.html`, 2026. ROS 2 Jazzy documentation. Accessed: 2026-08-30.

[28] J. Haviland and P. Corke, "Manipulator differential kinematics: Part ii: Acceleration and advanced applications," *IEEE Robotics & Automation Magazine*, pp. 2–12, 2023.

[29] G. Pahl, W. Beitz, J. Feldhusen, and K.-H. Grote, *Engineering Design: A Systematic Approach*. London: Springer-Verlag, 3rd ed., 2007. Traducción al inglés de la obra original en alemán *Konstruktionslehre*.

[30] M. Garreta Domingo and E. Mor Pera, *Diseño centrado en el usuario*. Barcelona: Fundació per a la Universitat Oberta de Catalunya, 2013. Material didáctico UOC. PID_00176058.

[31] D. A. Norman, *The Design of Everyday Things*. New York: Basic Books, revised and expanded ed., 2013. Primera edición publicada en 1988 como *The Psychology of Everyday Things*.

[32] T. Lowdermilk, *User-Centered Design*. Sebastopol, CA: O'Reilly Media, 2013.

[33] K. T. Ulrich and S. D. Eppinger, *Diseño y desarrollo de productos*. México, D.F.: McGraw-Hill/Interamericana Editores, S.A. de C.V., 5 ed., 2013. Traducción de la 5a. edición de *Product Design and Development*. Traducción: Jorge Humberto Romo Muñoz y Ricardo Martín Rubio Ruiz.